

PG-02
Programming
In Ada95

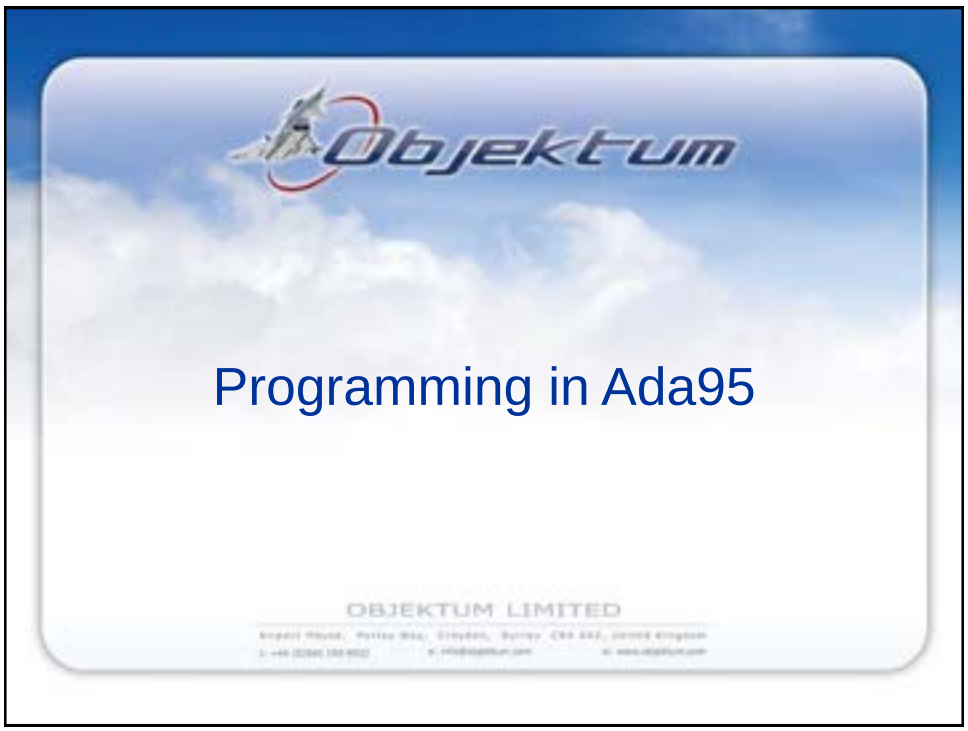
OBJEKTUM LIMITED
Airport House, Purley Way, Croydon, Surrey, CR0 0XZ, United Kingdom
t: +44 (0)845 199 9932 e: info@objektum.com w: www.objektum.com

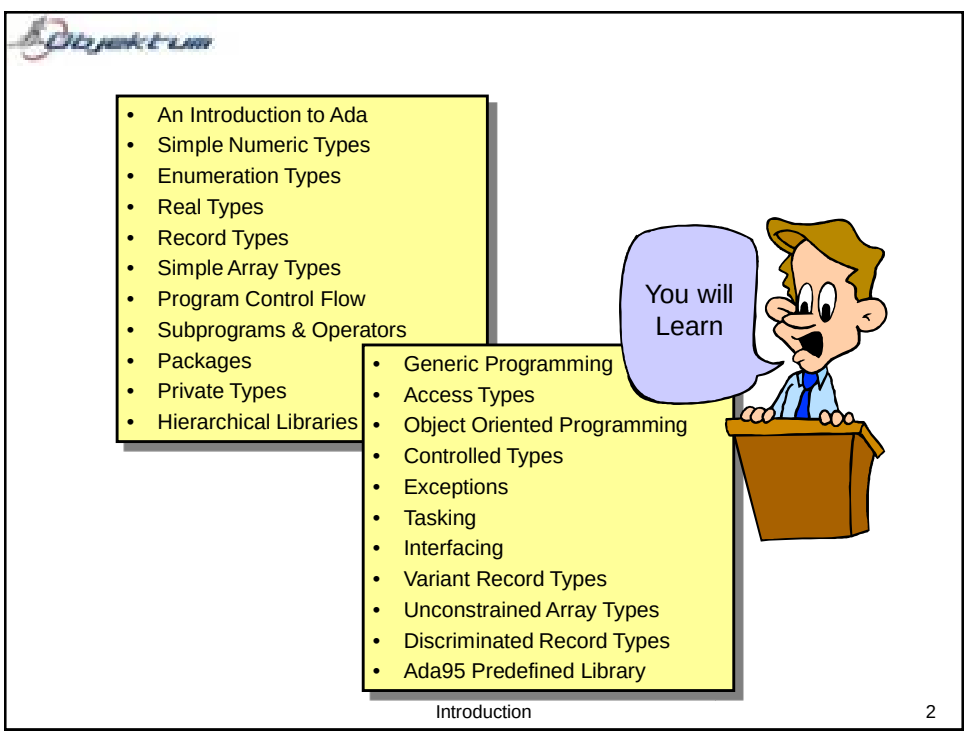
Contents

Introduction	3
An Introduction to Ada	11
Simple Numeric Types	23
Enumeration Types	31
Real Types	39
Record Types	45
Simple Array Types	49
Program Control Flow	55
Subprograms & Operators	65
Packages	73
Private Types	81
Hierarchical Libraries	89
Ada83 Generic Programming	95
Ada95 Generic Programming	105
Ada83 Access Types	113
Ada95 Access Types	121
Object Oriented Programming	131
Controlled Types	147
Ada83 Exceptions	151
Ada95 Exceptions	157
Ada83 Tasking	161
Ada95 Tasking	175
Interfacing	185
Variant Record Types	191
Unconstrained Array Types	195
Discriminated Record Types	199
Ada95 Predefined Library	203

© Objektum Ltd 2021

No part of this document may be reproduced, stored in a retrieval system or transmitted in any form or by any means; electronic, mechanical, photocopying, recording, scanning or otherwise without express written permission by Objektum Limited.


Objektum
Programming in Ada95
OBJEKTUM LIMITED


Objektum
An Introduction to Ada
Simple Numeric Types
Enumeration Types
Real Types
Record Types
Simple Array Types
Program Control Flow
Subprograms & Operators
Packages
Private Types
Hierarchical Libraries
You will Learn
Generic Programming
Access Types
Object Oriented Programming
Controlled Types
Exceptions
Tasking
Interfacing
Variant Record Types
Unconstrained Array Types
Discriminated Record Types
Ada95 Predefined Library
Introduction
2

GNAT Programming Studio



GNAT Programming Studio (GPS) is the GNAT Ada Integrated Development Environment (IDE).

GPS is developed by AdaCore and available in four different editions to meet the needs of different projects:

- Assurance – for projects that require certification
- Enterprise – for large scale industrial projects
- Developer – for teams that are new to Ada
- Community – free edition for non-commercial use

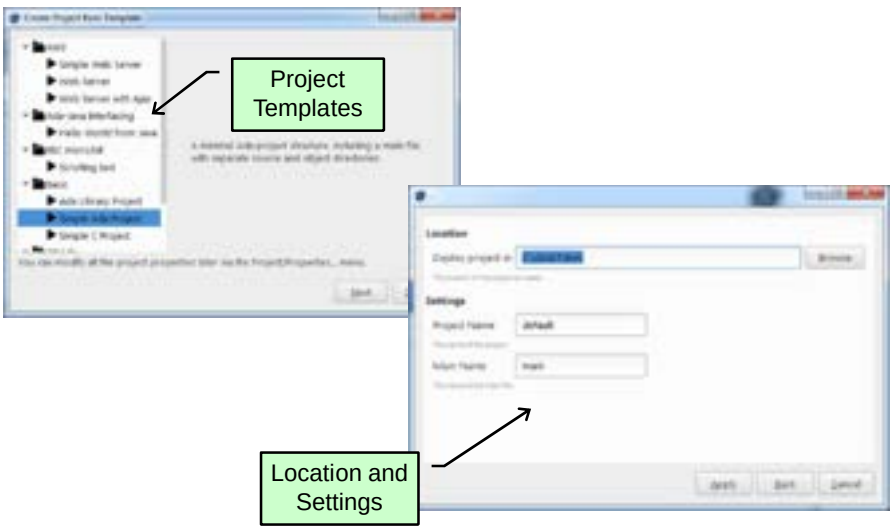
For more information on GPS go to www.adacore.com

Launching GPS

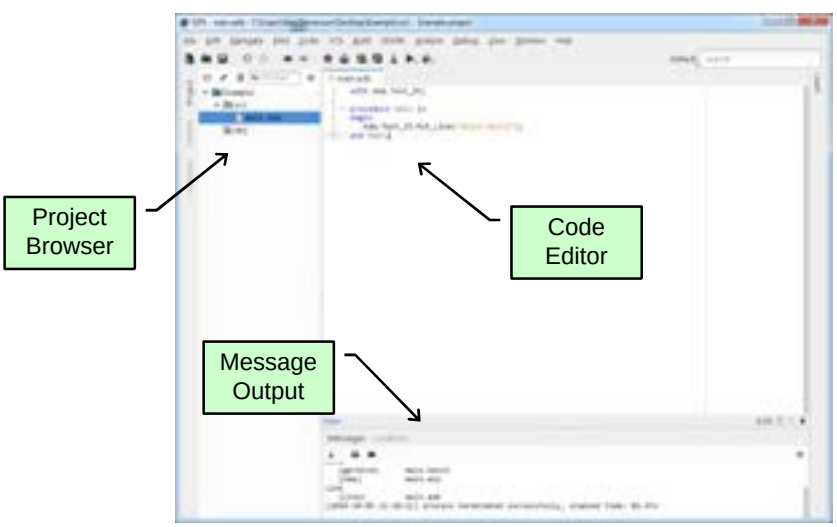
GPS can be launched using the desktop icon (if present) or from the Windows Start Menu.



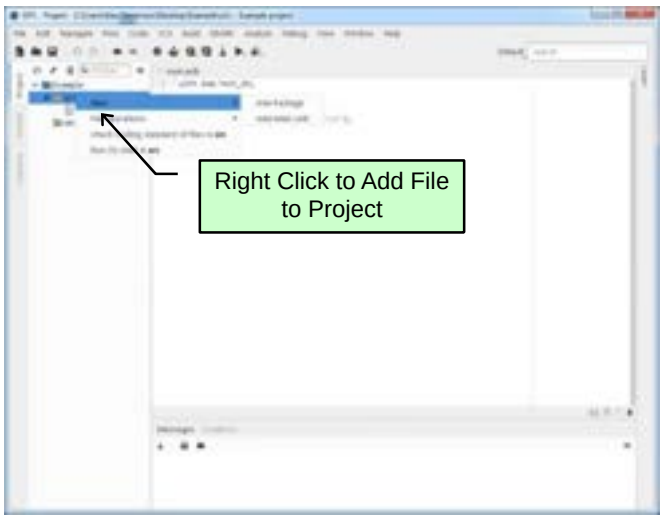
Creating New Projects



GPS Interface

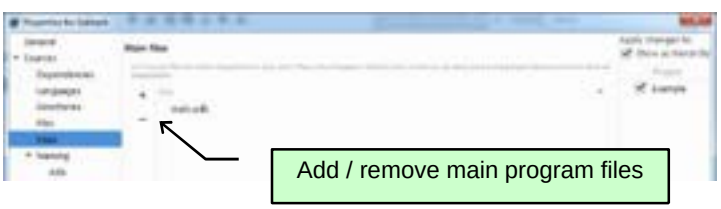


Adding Files to a Project

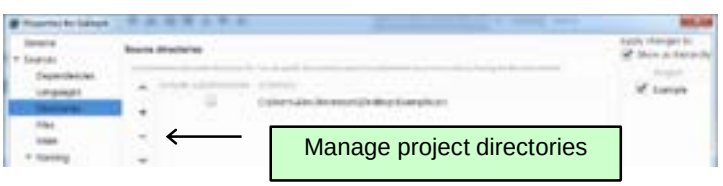


The Project Properties Window

To add or remove a main program open the project properties and select the Main option (**`Edit -> Project Properties -> Main`**)

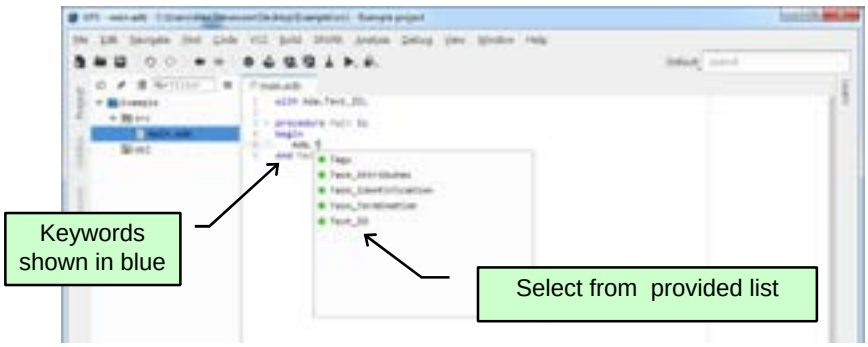


The project properties is also where directories may be added, removed and organised (**`Edit -> Project Properties -> Main`**



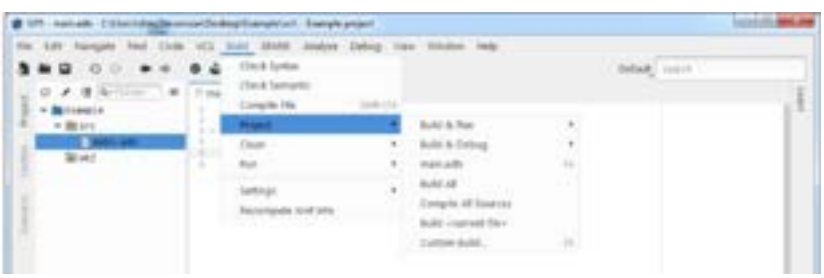
The Code Editor

The GPS code editor includes keyword highlighting and autocomplete features.



The Build Menu

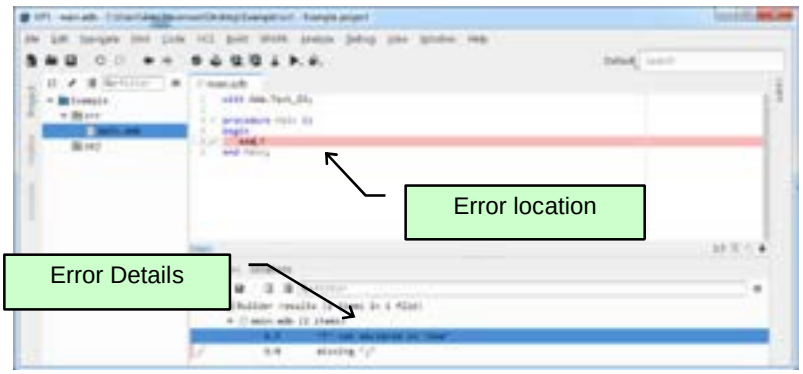
The Build menu provides various options for building, running and debugging code as well as other related functions.



The F4 key can also be used to perform a build and run operation.

The Message Window

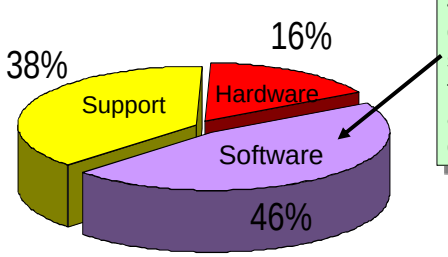
The Message Window provides information about build operations including errors found.



Selecting an error in the Message Window will highlight the line, in the code editor, where the error is located.

The History of Ada83

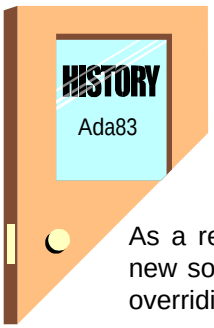
In 1973, the US Department of Defence (DoD) started to become concerned about the rising cost of developing application software



46% of overall system costs were being spent on software and 56% of those costs were attributable to embedded computer systems

Estimated DOD Annual Computer Costs

The History of Ada83



The DoD found that many of the problems stemmed from the large number of programming languages (over 500), many of these were either assembler or totally unsuitable.

As a result, the DoD commissioned the development of a new software language to be used on military projects, the overriding concerns were:

- Reliability
- Maintainability
- Portability
- Reusability



The History of Ada83

The new language was not finalised until 1983, when it achieved the ANSI standard, which is known as Mil Std 1815A.

The language was named Ada83 after Augusta Ada Byron, Countess of Lovelace.

Ada, the daughter of Lord Byron, was the assistant and patron of Charles Babbage and worked on the mechanical analytical engine.

In a very real sense she was the world's first programmer.



The History of Ada95

The DoD established the Ada 9X project in 1988 to revise the Ada language. The requirements for the new language were published in 1990.

It was deemed too costly to implement a language that met all the requirements in its entirety, on every conceivable architecture.

To eliminate uncontrolled subsets springing up, it was decided that Ada95 would consist of :

- New core language features
- Specialised application areas



An Introduction to Ada

Objective

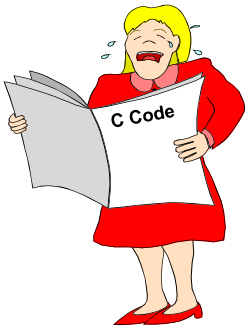


- To understand the Goals of the Ada Language
- To understand what makes up an Ada program, how they are developed and how a program should be laid out
- To understand the fundamental difference between Objects and data types and what predefined types are provided by the Ada language
- To Understand what pragmas and attributes are

Ada Language Goals

Addresses the important issues for generating high integrity software:

- Readability
- Encapsulation
- Modularity
- Strong Typing
- Parallel Processing
- Generic units
- Run-Time Error Detection
- Object-Oriented Programming (Ada95 Only)



Developing Ada Programs



Text_IO.Put ("Hello") ;

High Level Language

Compile

Linker

```
LD    A,100
OUT   (84),A
INC   A
JPNZ  2000
```

Object Code Assembler

Download

Executable Image

```
10100110
00110100
11100111
```



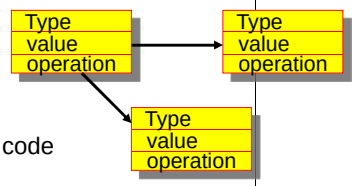
Target

Ada programs are written as lines of text in source files which are compiled, linked then possibly downloaded to the target computer

What makes up an Ada Program?

- Physical structure
  - A collection of source files
    - Packages (Specification and Body)
    - Procedures
    - Functions
- Logical Structure
  - Types defining values and operators
  - Procedures and functions containing code
- A running program
  - Executes functions and procedures
  - Creates and/or manipulates objects



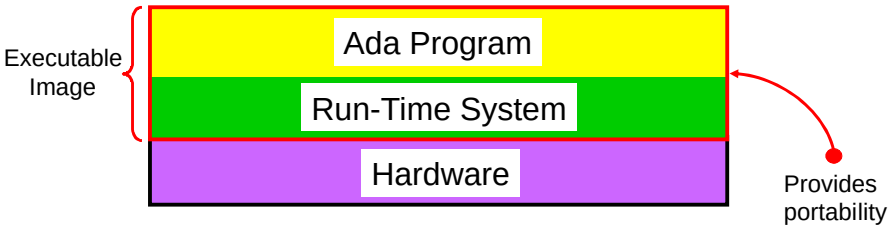




Ada Program Support

An Ada program is supported by a Run-Time System (RTS):

- Provides program support
- Provides an interface to the execution environment
- Is the key to program portability



Executable Program Structure

My First Program!



Main program starts at a procedure called Hello

Context clause, include facilities provided by the package Text_IO

Text_IO used for I/O

Statement terminated with a semicolon

```
with Text_IO;

procedure Hello is
begin
   Text_IO.Put ( “Hello World” ) ;
end Hello ;
```

define the start and end of a block

End of program execution

Reserved Words

Have special syntactical significance, those in RED are Ada95:

abort	declare	goto	package	separate
abs	delay	if	pragma	subtype
abstract	delta	in	private	tagged
accept	digits	is	procedure	task
access	do	limited	protected	terminate
aliased	else	loop	raise	then
all	elsif	mod	range	type
and	end	new	record	until
array	entry	not	rem	use
at	exception	null	renames	when
begin	exit	of	requeue	while
body	for	or	return	with
case	function	others	reverse	xor
constant	generic	out	select	

Identifiers

Uniquely identify program elements:

- Rules:
  - Must start with a letter
  - Possibly followed by one or more letters
  - Can contain isolated underscores
  - Can contain digits
  - Must not be one of the reserved words



Q. Which of the following are legal identifiers?



Ada
ZH_045
fish&chips
Aircraft-Height
Climb Rate
Message__time
procedure

Comments

Aid program understanding by someone other than the author:

- Rules:
  - Preceded by two hyphens (minus signs)
  - The two hyphens must be adjacent
  - Are written as arbitrary text
  - Not inserted in the middle of Ada statements
  - May be the only thing on the line
  - May follow an Ada statement



```
-- This is an example of a comment

procedure Hello is    -- This is the start of my program
begin
```

Expressions

Once an identifier has been declared, it can be used in an expression

- Actions are given symbols in Ada
  - Called operators
- Compiler checks all operators are performed on the correct type
- Ada is strongly typed



All identifiers MUST be declared before they are used ALWAYS

Lexical Elements

An Ada program is written as a sequence of lines of text containing:

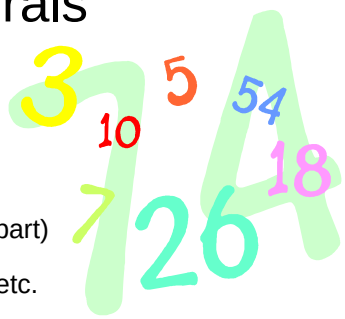
- The letters of the alphabet (upper and lower case)
- The digits 0 - 9
- Various special characters " # & ' ( ) * + , - . / : ; < = > _ |
- The space character

```
Key_Pressed := 6 ;   -- User has selected key 6
```

Identifier
compound delimiter
Numeric Literal
end of statement
comment

Numeric Literals

Represent numbers in the Ada program:

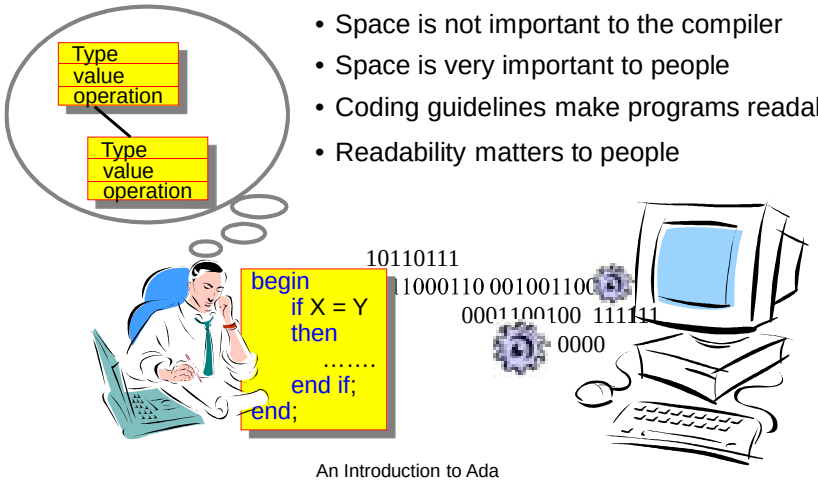


- Can be divided into 2 classes:
  - Integer literals (No decimal point)
  - Real literals (Contains a fractional part)
- Can be expressed in base 2, 8, 10, 16 etc.
- Can include underline (to aid readability)

```
0          --  zero (obviously)
21         --  twenty one
10_150     --  10,150 (Underline to aid readability)
16#10#     --  10 Hex  (16# denotes the base 16)
2#0101#    --  5 Binary (2# denotes base 2)
7E2        --  exponential = 700
```


Program Layout

Ada is a free format language:



- Space is not important to the compiler
- Space is very important to people
- Coding guidelines make programs readable
- Readability matters to people

Pragmas

Compiler with a directive which is not actually part of the program:



- Ada compilers must implement the defined set of pragmas but are free to provide their own:
  - Inline (operation) - Inline expansion required
  - Pack (type) - Minimum storage space required
  - Optimise (Time etc.) - Specifies compiler optimisation
  - Storage_Size (task) - Specifies the storage required for a task
  - Import (…) - Import external program entity
  - Suppress (identifier) - Omit run time checks (Not recommended)

Attributes

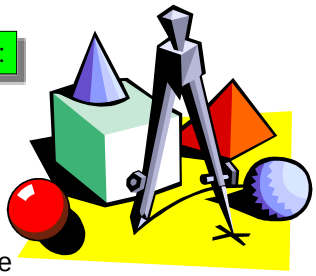
Can be used to provide a level of abstraction:



- Aids readability
- Are not reserved words
- Should be used to reduce maintenance
- Can be used to find bounds, ranges and even object sizes
  - Object'First - First element / Value
  - Object'Last - Last element / Value
  - Object'Range - First element to Last element
  - Object'Length - Number of elements
  - Object'Size - Size of the object (in bits)

What is an Object?

The following list helps to clarify the distinction:

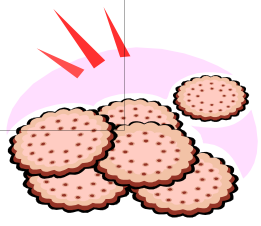


- Objects exist in Memory at Run-Time
- Values are stored in objects
- Objects hold a single value at any one time
- Objects are declared to be of a specific type
- Objects can be either:
  - Variable - Value may change during the life of the program
  - Constant - Has an initial value that never changes

What is a Data Type?

A Data Type can be described as:

- A template which defines the properties that will be possessed by objects declared of that type:
  - Possible set of values the object can take
  - Set of operations that can be performed
- All objects are declared to be of a specific type
  - Two objects of the same type can differ in value, but not in kind



Predefined Types

The are a number of types predefined by the Ada language :

- Integer - Permit values which are whole numbers
- Float - Permit real number values
- Duration - Fixed point type, represents periods in seconds
- Character - Enumeration type represents the character set
- String - Represents collections of characters
- Boolean - Represents Boolean values (True / False)



Predefined types are non portable and should be used with caution



Blocks, Scope and Visibility

Two fundamental rules of Ada should be noted:



- Any item must be declared before it is used
  - Procedure & function
  - Types and Objects
- Objects disappear as soon as their scope is left
- Objects and types can be declared in any declarative region
- A declare block can be used to declare types and objects
- Anything declared in a block is only visible inside that block

Text I/O Facilities

Ada does not provide any special language features for I/O

- Its approach is to provide I/O by means of a set of predefined library packages which include:
  - File management operations
  - Default I/O files
  - Column, Line and Page controls
  - Input/Output for Integer Types
  - Input/Output for Real Types
  - Input/Output for Enumeration Types



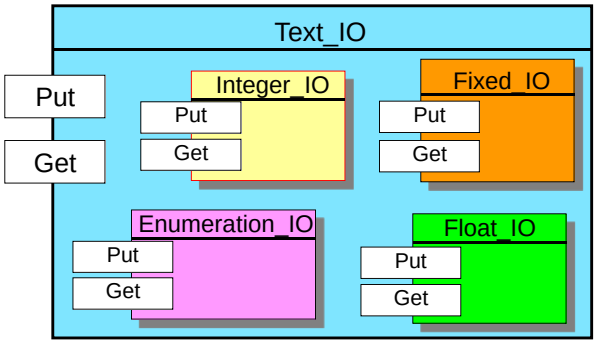
Text_IO is a predefined package that deals with information that is in human readable format, i.e. characters and strings.

Text I/O Facilities

Text_IO contains internal various packages:



- These internal packages are “Generic” to allow their reuse for the numerous types that can be defined in any system



Text I/O Examples

Integers and floating point numbers can be input and displayed by instantiating the appropriate generic package:

```
with Text_IO;
procedure Print_Numbers is
      package My_Integer_IO is new Text_IO.Integer_IO (Integer);
      package My_Float_IO is new Text_IO.Float_IO (Float);
begin
      My_Integer_IO.Put  (Item => 748,
                          Base => 16,
                          Width => 8); -- Will print “16#2EC#<space>

      My_Float_IO.Put (Item   => 996.0);        -- Will print 996.0
end Print_Numbers ;
```

Simple Numeric Types

Objective



- To understand what Ada discrete types are
- To understand what predefined types are provided and why they make a program non portable
- To understand how to create user defined numeric types and declare objects
- To Understand what subtypes are, how to use them and how to perform type conversion

What are Discrete Types?

Permit values which are either:

- Whole numbers (Integers)
  - Signed
  - Unsigned
  - Modular - Ada95 Only
- Named values (Enumeration types)

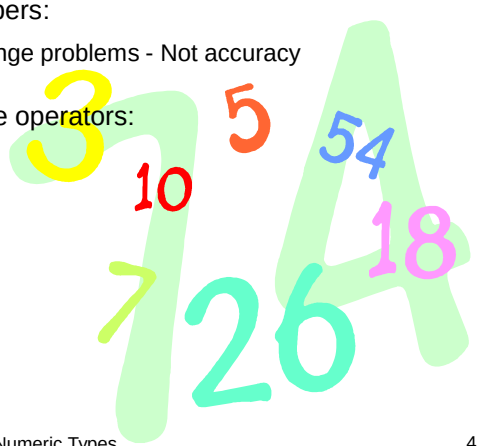


Discrete types are characterised by the set of values and the set of operations that can be performed on that type

What are Integers?

Integers can be characterised as:

- Representing whole numbers:
  - Only suffer from range problems - Not accuracy
- Providing a set of primitive operators:
  - Addition
  - Subtraction
  - Multiplication ……
- Can be expressed as:
  - Decimal
  - Binary
  - Hexadecimal ……



Predefined Integer Types

All implementations of Ada have predefined integer types:

16 bits on 16 bit machine

```
type Integer is range -2**15 .. 2**15-1;
type Long_Integer is range -2**31 .. 2**31-1;
```

32 bits on 16 bit machine

FREE!

32 bits on 32 bit machine

```
type Integer is range -2**31 .. 2**31-1;
type Long_Integer is range -2**63 .. 2**63-1;
```

64 bits on 32 bit machine

Declaring Objects

A variable is created by means of a declaration:



Introduces a new variable 'Counter'

```
Counter: Integer;
```

?

Integer

Multiple variables can be declared :

Introduces 2 new variables

```
Counter_1, Counter_2 : Integer;
```

? ?

More commonly, the declarations are separated and declared on separate lines

Using Objects

A variable may be assigned an initial value:

Introduces a new variable 'Counter' with an initial value of 1

```
Counter : Integer := 1;
```

→ 1

Variables are assigned values during program execution:

```
Counter := 2;
```

→ 2 — Counter now contains '2'

```
Counter := Counter + 1;
```

→ 3 — Add '1' to the value of counter

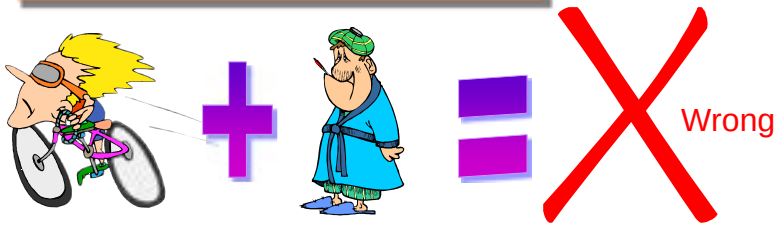
Note: ':=' can be followed by an expression provided it "produces a result of the correct type"

Using Predefined Types

Is bad programming practise, consider:

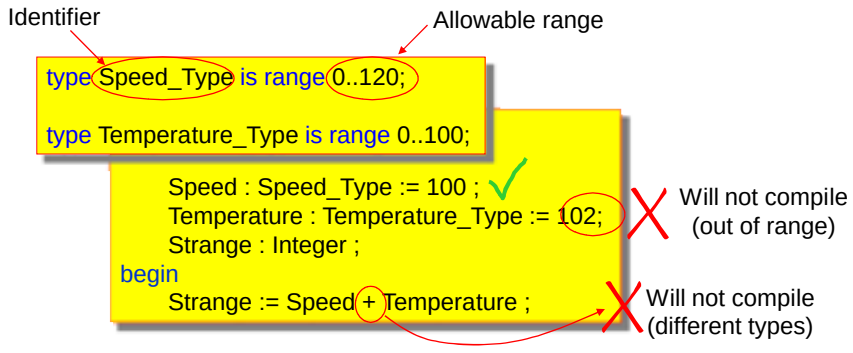
```
declare
      Speed : Integer := 100 ;
      Temperature : Integer := 102 ;
      Strange : Integer := Speed + Temperature ;
begin
      ……
```

Adding Speed to Temperature does not make any sense, but it would compile



Declaring Integer Types

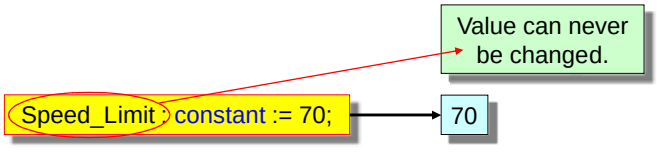
Allows the compiler to check calculations and assignments are being performed on the correct objects:



Declaring Constants

Constants are declared in a similar way to variables, but:

- Have the reserved word constant in their declaration
- Must be initialised when declared



This is known as a number declaration and simply provides a name for a number:

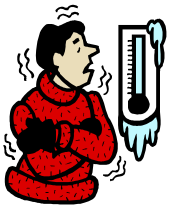
- Is considered to belong to an anonymous type “Universal Integer”.
- The range of Universal Integer is essentially infinite

Declaring Constants

Constants should be declared of the appropriate type:

- Avoids accidental mixing of types
- Misuse of a constant will be detected by the compiler

```
Top_Speed : constant Speed_Type := 70;
Body_Temperature : constant Temperature_Type := 36;
```



```
      Speed : Speed_Type := Top_Speed ;
      Temperature : Temperature_Type ;
begin
      Temperature := Top_Speed ;
```

Will not compile (different types)

Type Conversion

Can override the strong Ada typing:

Types to represent speed and temperature

```
declare
      type Speed_Type is range 0..70;
      type Temperature_Type is range 0..100;

      Speed : Speed_Type := Speed_Type'Last;
      Temperature : Temperature_Type := 100;
begin
      Temperature := Temperature_Type (Speed);
      Speed := Speed_Type (Temperature);
```

Attribute

Q. What will happen here?

Convert Speed to Temperature_Type and assign it to object Temperature

What are Subtypes?

A subtype defines a set of values for some other type:

For example if we were required to manipulate dates, the day must lie in the range of 1 to 31, we could write:

```
type Day_Type is range 1..31;
…….
Today    : Day_Type := 10;
Tomorrow : Day_Type := 11;
```

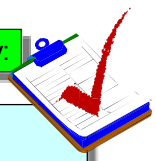


We could then further restrict the number of days in each month:

```
subtype Feb_Day_Type is Day_Type range 1..29;
```


Subtype Rules

When assigning subtypes, strong Ada typing rules still apply:



- Both sides must be of the same type
- The assignment must satisfy any constraints
  - if not, the assignment is ignored and an exception is raised

- The compiler can detect the violation of the first rule
- Violation of the second rule will be detected at run time

```
The_Date : Day_Type;      → Range 1..31
Feb_Date : Feb_Day_Type;  → Range 1..29
……
The_Date := Feb_Date;     ✓ → Will always work
Feb_Date := The_Date;     ✗ → Will fail if The_Date is 30 or 31
```



Predefined Subtypes

In addition to the predefined integer type, there are also two useful predefined subtypes :

```
subtype Natural is Integer range 0..Integer'Last;
subtype Positive is Integer range 1..Integer'Last;
```

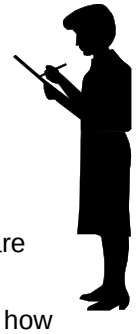
Attributes

Sensible use of subtypes has two advantages:

- It can ensure that programming errors are detected earlier by preventing objects from being assigned inappropriate values
- It can increase the execution efficiency of the program

Enumeration Types

Objective



- To understand what enumeration types are and how to define them
- To understand what attributes can be used with enumeration types and what membership tests are
- To Understand the predefined type Boolean and how to use it
- To understand how to print enumeration values

What are Discrete Types?

Permit values which are either:

- Whole numbers (Integers)
  - Signed
  - Unsigned
  - Modular - Ada95 Only
- Named values (Enumeration types)

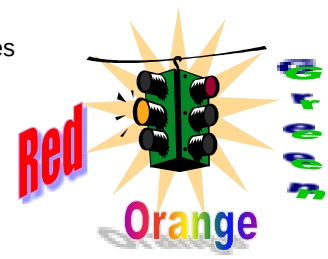


Discrete types are characterised by the set of values and the set of operations that can be performed on that type

What are Enumeration Types?

Enumeration types can be characterised as:

- Listing the possible values by literals
  - No limit to the number of literal values
- Literals are expressed as:
  - Text
  - Must be identifiers
  - Case insensitive
- Providing a set of operators & attributes:
  - < , <= , > , >=, =, /=
  - First , Last , Succ , Pred ….



Declaring Enumeration Types

Same as integers, except explicitly listed by literal value:

```
type Day_Type is  (Monday, Tuesday, Wednesday,
                   Thursday, Friday, Saturday, Sunday);

type Weekend_Type  is (Saturday, Sunday);
```

Overloaded



When referring to the literals, qualification can be used to uniquely identify the type:

Unique

Qualification

```
Work_Day : Day_Type := Monday ;
Church_Day : Weekend_Type := Weekend_Type'(Sunday);
```



Subtypes of Enumeration Types

Can be defined in the same way as Integer subtypes:

- Defines a set of values for some other type
- Are considered to be the same type



```
type Day_Type is  (Monday, Tuesday, Wednesday,
                   Thursday, Friday, Saturday, Sunday);

subtype Weekend_Type  is Day_Type range Saturday .. Sunday;
```



Care should be taken when defining subtypes of enumeration types:

- Changes to the base type, could affect the subtype

Membership Tests

Determine whether a value is within a specified range

- Apply to all scalar types
- Include constraints implied by subtypes

In - Returns True if value is in range
not in - Returns True if value outside the range

```
type Day_Type is  (Mon, Tues, Wed, Thurs, Fri, Sat, Sun);
subtype Weekday_Type is Day_Type range Mon .. Fri;

if Today in Weekday_Type
if Counter not in 10..25
```

→ False for days at the weekend
→ True if counter outside 10 to 25

Discrete Type Attributes

Return information about types and/or objects:

- Attributes are extremely useful and very simple to use.
- Should be used wherever possible to aid maintenance

```
type Vehicle_Type is (Car, Truck, Motorbike) ;
type Speed_Type is range 0..120;

Vehicle_Type'First;
Speed_Type'Last;
Vehicle_Type'Succ (Car);
Speed_Type'Pred (70);
Vehicle_Type'Pos (Truck);
Vehicle_Type'Val (2);
```

→ [= Car]
→ [= 120]
→ [= Truck]
→ [= 69]
→ [= 1]
→ [= Motorbike]



Boolean Types

Is a predefined enumeration type and can be considered as:

```
type Boolean is (False , True);
```

- A Boolean result is produced by the operators =, /=, <, <=, >, >=
- Is used in constructions such as if statements:

if Missile = SRAM	True if the Missile is a SRAM
if Today /= Sunday	True for days except Sunday
if Counter > 50	False if counter is 50 or less
if Speed <= 70	False if you are speeding

Boolean Operators

There are additional operators that apply to booleans:

```
not   -- Unary operator, changes True to False and vice versa
and   -- Binary operator, result is True if both operands are True
or    -- Binary operator, result is True if one or both operands are true
xor   -- Binary operator, result is True if either operand (not both) are true
```

Truth tables for and, or and xor

and	F	T
F	F	F
T	F	T

or	F	T
F	F	T
T	T	T

xor	F	T
F	F	T
T	T	F



Boolean Operator Precedence

or, and, xor are equal, but are lower than any other operator, e.g. =

- Although the precedence is equal:
  - and, or and xor cannot be mixed up without parentheses

B and C or D → is illegal (will not compile)

B and (C or D) → is legal



Calculate the result if:

B => True	B => True
C => False	C => False
D => True	D => False

Text I/O Facilities



Are provided for Enumeration types:

```
with Text_IO;
.....
      type Colour_Type is (Red , White , Blue);      -- Enumeration Type
      Colour : Colour_Type := Red;                   -- Object declaration

      package Colour_IO is new Text_IO.Enumeration_IO (Colour_Type);
      package Boolean_IO is new Text_IO.Enumeration_IO (Boolean);
begin
      Colour_IO.Put (Colour);  -- “RED” will be displayed on the screen
      Colour_IO.Get (Colour);  -- Read an enumeration from the keyboard
      Boolean_IO.Put (True);
```

The Image Attribute

Is an easier method of displaying the values of discrete types



```
type Vehicle_Type is (Car, Truck, Motorbike) ;
type Speed_Type is range 0..120;
Speed : constant Speed_Type := 25;
begin
Text_IO.Put (Vehicle_Type'Image (Vehicle_Type'First)) ;
Text_IO.Put ( Speed_Type'Image (Speed)) ;
Text_IO.Put ( Vehicle_Type'Image (Car)) ;
Text_IO.Put ( Speed_Type'Image (Speed_Type'Last)) ;
Text_IO.Put (Integer'Image (Vehicle_Type'Pos (Truck)) ;
```



Q. What will be displayed...

Intentionally Left Blank

Real Types

Objective



- To understand what real types are and the predefined floating point type
- To understand how to define floating point types
- To Understand how to define fixed point types

What are Real Types?

Integer types are exact, Real types have a decimal point:

- Real types can only be represented approximately
- Real types are subdivided into two categories:
  - Floating Point Types
    - Which have a relative error proportional to the magnitude of the value
  - Fixed Point Types
    - Which have an absolute error that is constant



Floating Point Types

All implementations of Ada have the predefined type Float

- Some implementations have other floating point types such as *Long_Float*, *Short_Float*
- As with Integers, it is good practice not to use the predefined types but to define your own



```
type My_Float_Type is digits 7;
```

Defines a type (based on a predefined floating point type) with at least 7 decimal digits of precision.

Must be static

The actual precision and range supported depends on the hardware.

Floating Point Types



It is possible to declare a constrained type

7 digits of precision — Must be static — Range 0.0 to 150.0.

```
type Speed_Type is digits 7 range 0.0 .. 150.0;
```

- It is possible to declare a constrained object or subtype:

Dynamically constrained object

```
Moped : Speed_Type range 0.0..45.0;
subtype Legal_Speed_Type is Speed_Type range 0.0..70.0;
```



If the legal speed limit is violated or the moped goes too fast, then a Run-Time error will be raised.

Floating Point Type Notes

- The actual implemented range is known as the base range
- The actual precision is known as the base decimal precision
  - The precision may not hold over the complete base range
  - Ada guarantees a range over which the precision is maintained:
    - This is known as the safe range

```
-10.0**(4*D) .. +10.0**(4*D)
```

Where D is the decimal digits requested



Fixed Point Types

Are normally used in specialist application areas:

- For performing calculations on machines without floating point capability
- An ordinary fixed point type declaration specifies an absolute error and a mandatory range



```
type My_Fix_Type is delta Step range Min .. Max;
```

Accuracy
(must be *Real* & Static)

Range
(must be *Real* & Static)

Fixed Point Types

Are characterised by two attributes:

- Size - The number of bits required to hold the number
- Small - The absolute precision

-1.0 to +1.0, accuracy of 0.1.

Size will be 4
Small could be 1/16 (0.0625)

```
type Fix_Type is delta 0.1 range -1.0 .. +1.0;
```



The model number would be:

$-^{15}/_{16}$, $-^{14}/_{16}$ …, $-^{1}/_{16}$, 0 , $+^{1}/_{16}$, …, $+^{14}/_{16}$, $+^{15}/_{16}$

Fixed Point Types

We can override the rule that small is a power of 2:

- By using the representation clause:

```
for Fix_Type'Small use 0.1;
```

Small is now exactly 0.1



The model number will now be:

….. -0.2 , -0.1, 0 , 0.1 , 0.2, …,

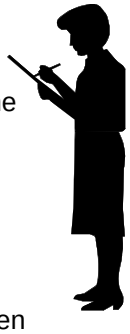
- If small is explicitly declared, then any spare bits will provide extra range and not accuracy.
- The advantage of using the default power of 2 is that conversion between fixed point types and other numeric types can be based on shifting (which is run time efficient).



Intentionally Left Blank

Record Types

Objective



- To understand what records are and how to define them
- To understand how to assign values and access record components
- To understand what aggregate assignment is when applied to records

What are Records?

Sets of logically grouped components, typically of different types:

- Data is often grouped together, eg
  - Dates can be represented DD/MM/YY
  - A person can be described by name, date of birth, etc
- A new composite type can be defined using a *record*
  - record corresponds to *struct* in C and C++



```
type Date is
record
      Day, Month , Year : Natural ;
end record;
```

```
type Person is
record
      Name : String (1..5);
      DOB : Date ;
end record ;
```


Objects of Record types

An object can be declared (and initialised if required):

```
The_Date : Date := (Day      => 28 ,
                    Month    => 07 ,
                    Year     => 1964);

The_Person : Person := (Name  => "Clive" ,
                        DOB   => The_Date);
```

Aggregate assignment recommended for records assignment

Record within a record

To access the record components, dot notation is used:



```
Text_IO.Put (The_Person.Name);
The_Person.DOB.Day := 01 ;
```

Record within a record

Records are Definite Types

And can therefore be array components:



```
type People is array (1..3) of Person ;

The_People : People :=
        (1 => (Name      => "Mary " ,
               DOB   => (Day    => 01 ,
                         Month => 04 ,
                         Year   => 1974)),
         2 => ….));
```



Work out the following:

- Print the name in array element 2
- Set the Month Mary was born to July
- Print Mary's details on the screen

Intentionally Left Blank

Simple Array Types

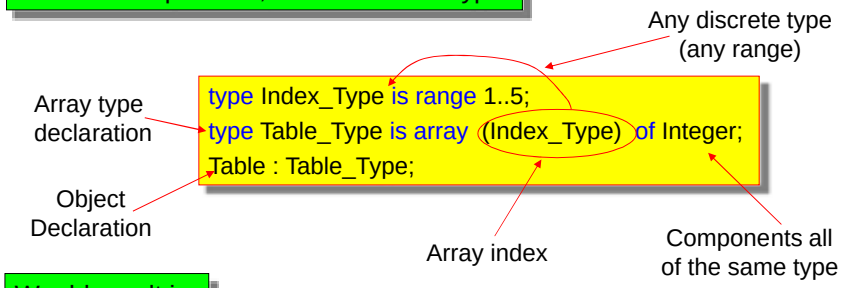
Objective



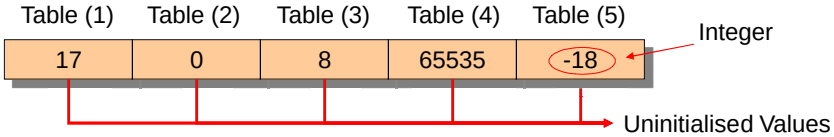
- To understand what arrays are and how to define them
- To understand how to assign values and access array elements
- To understand what attributes can be applied to arrays and what aggregate assignment is
- To Understand how to define and use multidimensional arrays and arrays of arrays

What is an Array?

A set of components, all of the same type:

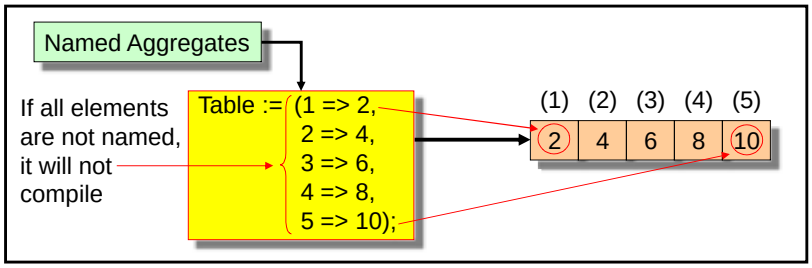


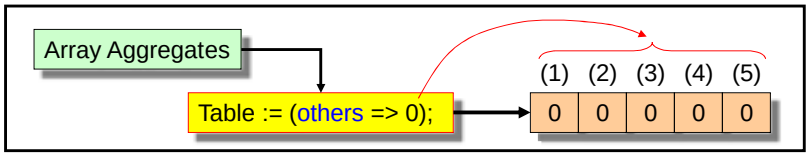
Would result in:



Array Assignment

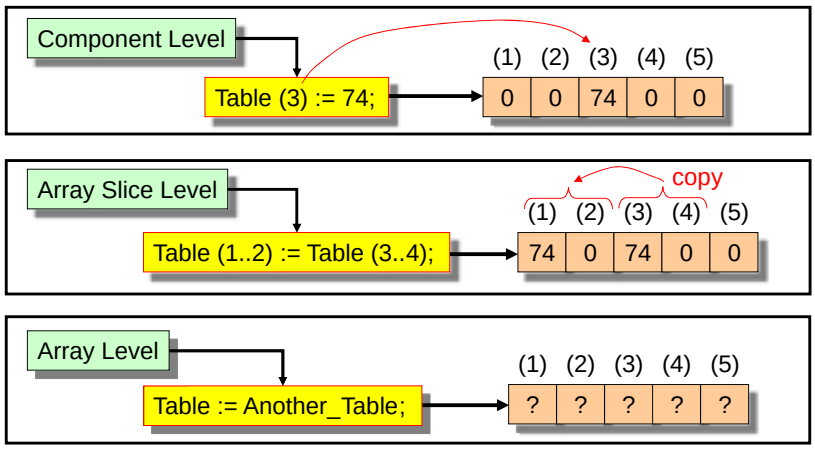
Can also be used in conjunction with the declaration:





Accessing Arrays

There are various methods of accessing arrays:



Anonymous Objects

Can be declared, i.e. with no associated types:

Anonymous Objects

```
Object_1 : array (1..6) of Boolean;
Object_2 : array (1..6) of Boolean;
Object_1 := Object_2;
```

Will NOT Compile:
Even though they look the same, they are actually different types

We cannot assign (or test) the objects

We could however write:

Objects now of the same type

```
type Array_Type is array (1..6) of Boolean;
Object_1 : Array_Type;
Object_2 : Array_Type;
Object_1 := Object_2;
```

Now allowed

Constant Arrays

Must have an initial value for each component:

- Constant arrays are most often used as lookup tables and provide an efficient way to determine constant values



```
type Ordnance_Type  is (Bomb, AAMRAM, SRAM , Sidewinder) ;

Ordnance_Is_Missile : constant array
      (Ordnance_Type) of Boolean := (False, True , True , True) ;
```

Discrete type (enumeration)

Constant, therefore must have initial value

Positional aggregation?

Array Aggregates

Positional aggregation can cause a maintenance overhead:

- Especially if the index is an enumeration which is re-ordered

Bomb is a missile, AAMRAM is not

```
type Ordnance_Type  is (AAMRAM , Sidewinder , SRAM, Bomb) ;

Ordnance_Is_Missile : constant array
      (Ordnance_Type) of Boolean := ( False, True , True , True ) ;
```

Named aggregation avoids this problem:

Don't have to be in order

```
Ordnance_Is_Missile : constant array (Ordnance_Type) of Boolean :=
                  (AAMRAM     => True ,
                   Sidewinder => True ,
                   SRAM       => True ,
                   Bomb       => False) ;
```

Array Attributes

Arrays have various attributes relating to their indexes.



```
type Key_Type is range 1..20;
Active_Keys : array (Key_Type) of Boolean;
……
Active_Keys'First
Active_Keys'Last
Active_Keys'Length
Active_Keys'Range
```

- Active_Keys'First → Returns 1, i.e. first array index
- Active_Keys'Last → Returns 20, i.e. last array index
- Active_Keys'Length → Returns 20, i.e. Length of the list
- Active_Keys'Range → Returns a range 1 .. 20

For example:

1

```
Active_Keys (Active_Keys'First) := True;
```

Sets element 1 = True

Multi-Dimensional Arrays

Have several dimensions, a range is provided for each dimension:

```
Object : array (1..4 , 1..5) of Float ;
```

The components can then be accessed as before but with the addition of another dimension:

```
Object (2 , 4) := 0.0 ;
```

	1	2	3	4	5
1	1.7	4.2	7.6	4.1	9.6
2	9.7	1.1	4.0	0.0	2.3
3	3.6	6.4	2.9	10.2	41.1
4	8.4	0.8	21.8	13.9	12.1

The range of the index can be variable (i.e. not static) :



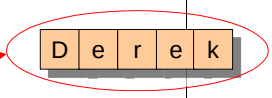
```
Index : Integer := 6;
Object : array (Integer range 1 .. Index) of Float;
```

Arrays of Arrays

An array component can be any definite type:

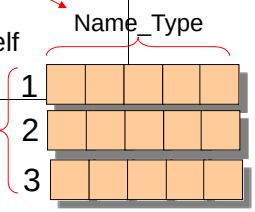
- A definite type being one for which an object can be declared

```
subtype Name_Type is String (1..5);
Name : Name_Type := “Derek”;
```



- A fixed length array is a definite type:
  - Therefore can be an array component itself

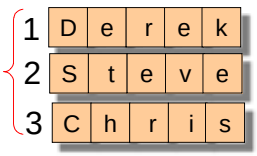
```
type People is array (1..3) of Name_Type;
```



Accessing Arrays of Arrays

Aggregates for multi-dimensional and arrays of arrays are identical:

```
The_ People : constant People :=
        (1 => “Derek“ ,
         2 => “Steve“ ,
         3 => “Chris” );
```



Work out the following:

Array of array access




```
Letter := People (2)  (4) ;     -- Value of Letter?
                                -- What type is Letter?
Name := People (3) ;            -- Value of Name?
                                -- What type is Name?
                                -- Change Derek to John
```

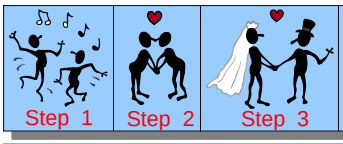
Program Control Flow

Objective



- To understand how to write programs with conditional branches using the ‘if’, ‘else’ and elsif’ statements
- To understand how to write a ‘case’ statement
- To understand how to write programs containing iteration using loops (‘for’ and ‘while’)
- To understand the ‘goto’ and ‘exit’ statements and why they should be avoided

Program Control Flow



Sequential

- Statements executed in sequential order
- Simple and compound statements



Selection

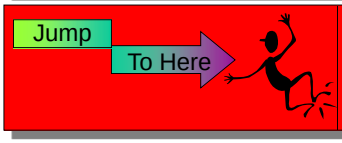
- Only one choice of several is executed
- *if* and *case* statements



- **Continuous**
- **Until I say so**
- **Number of times**

Iteration

- The same code is executed repeatedly
- *loop, while* and *for* statements



Transfer

- The point of execution jumps to another
- *exit* and *goto* statement

Program Control Flow

There are three main program control flow structures which are sufficient to write programs with discernible control flow:

```
if expression
then
    ......
end if ;
```

```
case identifier is
    ......
    ......
end case ;
```

```
loop
    ......
    ......
end loop ;
```

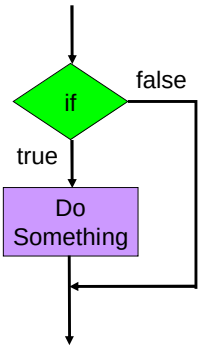


Each flow control structure exhibits a bracketing style, i.e. an opening reserved word, matched at the end by the same reserved word preceded by end

The *if* Statement



The simplest selection statement




```
if {boolean expression}
then
        statements ;
end if ;
```

Executes if result of expression is TRUE

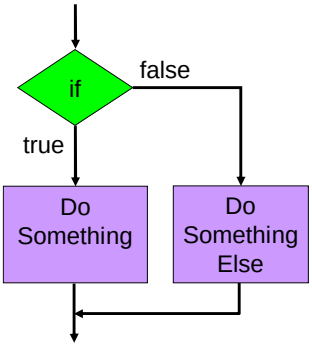
```
Text_IO.Get (Input) ;
if Input < 0
then
        Text_IO.Put ( “Is negative”) ;
end if ;
```

Note: use of {if .. end if}

The *else* Statement



A selection statement with an alternative path




```
if {boolean expression}
then
        statements ;
else
        statements ;
end if ;
```

Executes if result of expression is TRUE

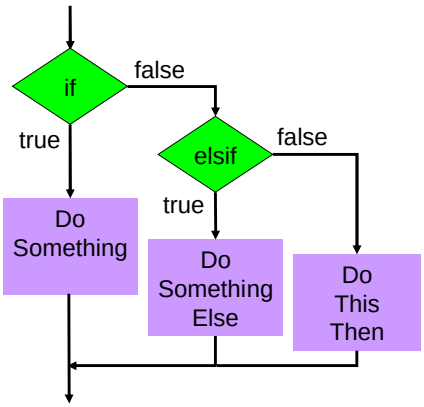
Executes if result of expression is FALSE

```
Text_IO.Get (Input) ;
if Input < 0
then
        Text_IO.Put ( “Is negative”) ;
else
        Text_IO.Put ( “Is positive”) ;
end if ;
```

The *elsif* Statement



A selection statement with an alternative selection path




```
if {boolean expression}
then
      statements ;
elsif {boolean expression}
then
      statements ;
else
      statements ;
end if ;
```

Only evaluated if first expression is FALSE

else part is optional

Compound *if* Statements



A compound selection statement

```
if {boolean expression}
and {boolean expression}
then
      statements ;
end if ;
```

Can be logical 'and' or 'or'

```
if {boolean expression}
or {boolean expression}
then
      statements ;
end if ;
```

Good, it's Saturday and the wife is away



```
if  Today = Saturday
and  The_Wife_Is_Here
then
   Go_Shopping ;
else
   Play_Football ;
end if ;
```

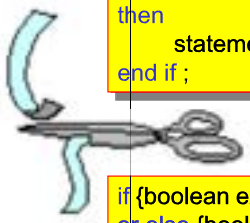
Selection Shortcuts



A compound selection statement with shortcuts

```
if {boolean expression}
and then {boolean expression}
then
        statements ;
end if ;
```

Expression **only** evaluated if the first expression is TRUE



```
if {boolean expression}
or else {boolean expression}
then
        statements ;
end if ;
```

Expression **only** evaluated if the first expression is FALSE

The *case* Statement

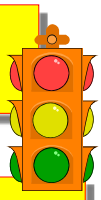


Multiway selections can be done with a *case* statement

```
case Identifier is
        when value_1 => statement ;
        when value_2 => statement ;
                        statement ;
        ……
        when others => statements ;
end case ;
```

'others' is required if not all possible paths are covered

```
type Light_Type is
   (Red , Orange , Green);
Traffic_Light : Light_Type ;
```



```
case Traffic_Light is
        when Red    => Stop ;
        when Orange => Ready ;
        when Green  => Go ;
end case ;
```

The *case* Statement



Monday or Tuesday

Order of label is irrelevant

Wednesday thru Friday

```
case Day is

    when Monday | Tuesday      => Do_Boring_Course ;
    when Wednesday .. Friday   => Sit_Around_Chatting ;
    when Saturday              => Go_Shopping ;
    when Sunday                => null ;

end case ;
```

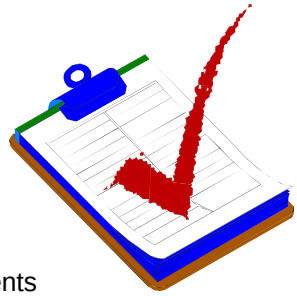
All alternatives covered, hence no 'others'

Must be static

If no action is required then the null statement must be used

case Statement Notes

- 'When' labels must be compile time constants (static)
- All possible alternatives must be covered
  - If not, a compiler error will be generated
  - 'others' can be used to catch alternatives
- Alternatives can have:
  - A single statement
  - Multiple statements

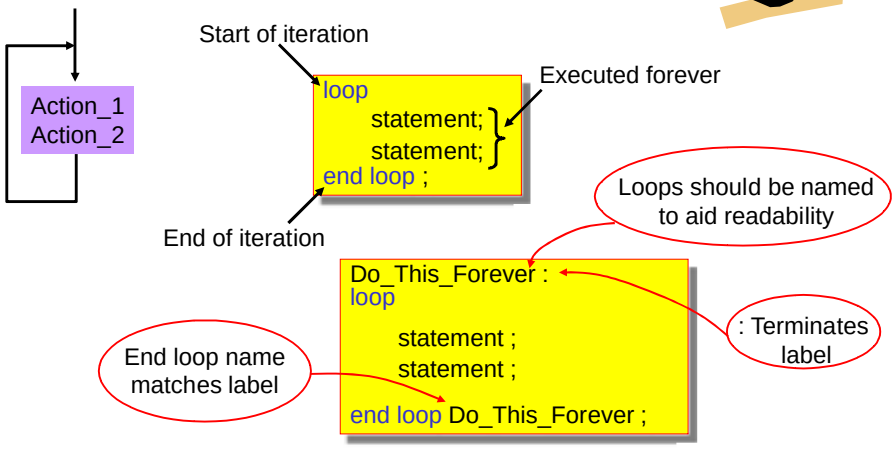


- More efficient than using multiple if statements
- if 'others' is used, it must be last and on its own
- Each alternative ends when the next 'when' statement is encountered

Indefinite Loops



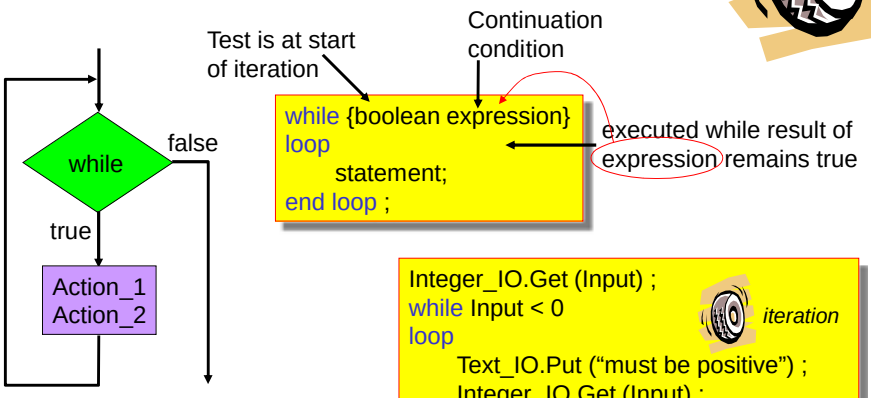
The simplest iterative statement (indefinite)



The *while* Statement



The conditional iterative statement



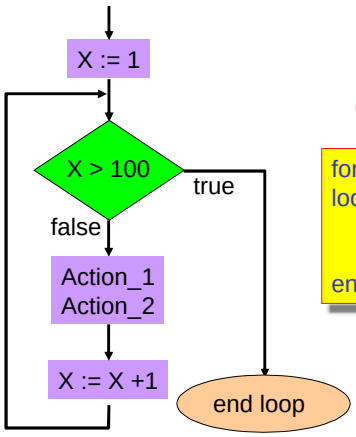

```
Integer_IO.Get (Input) ;
while Input < 0
loop
      Text_IO.Put ("must be positive") ;
      Integer_IO.Get (Input) ;
end loop ;
```

iteration

The *for* Statement



The most common iterative statement



X is declared automatically and only exists for the duration of the loop

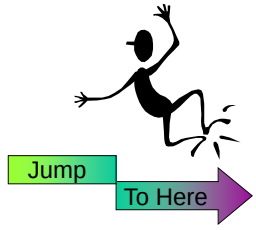
```
for X in 1 .. 100
loop
      statement ;
      statement ;
end loop ;
```

X can be any discrete type, e.g. enumeration type, integer

The loop can iterate in reverse by inserting the reserved word reverse after 'in' but before the discrete range

The *goto* Statement

Can be used to redirect the execution path






```
I : integer := -1 ;
<<start_of_loop>>
if i  >=  0  goto  end_of_loop;
Text_IO.Get (I) ;
goto  start_of_loop;
<<end_of_loop>>
```





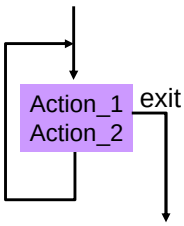
A *goto* statement shows poor logic; consider rewriting. It is prohibited in most organisations

The *exit* Statement



The simplest iterative statement (indefinite)

Jump To Here




```
Forever :
loop
      statement;
      statement;

      exit Forever
            when {boolean expression} ;

end loop Forever ;
```

Note label

Loop exits when Boolean expression evaluates to TRUE



A *exit* statement shows poor logic, consider rewriting.

Intentionally Left
Blank

Subprograms & Operators

Objective

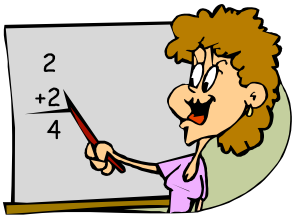


- To understand what functions and procedures are and how to define and call them
- To understand how to define and pass subprogram parameters (functions & procedures)
- To understand what default parameters are and how to use named parameter association
- To Understand what subprogram overloading is
- To understand how to define operators (+, -, / etc) and why we need to

Defining Functions

Functions always returns a result:

- The result is returned:
  - To be of a specific type
  - When a *return* statement is executed



Local declarations

Designator

Parameter(s)

Type returned

```
function Add (X , Y : in Integer) return Integer is
   Answer : Integer ;
begin
  Answer := X + Y ;
  return Answer ;
end Add;
```

Implementation

What does this *function* do?

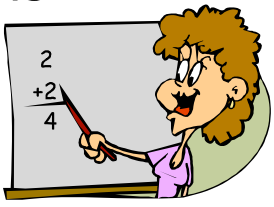
Must execute a *return* statement

End of Add

Calling Functions

The return value cannot be ignored:

- The result can be:
  - Assigned to an object
  - Used as part of an expression



Result of function call is assigned to an object which must be of the correct type

Result of function call is used as part of an expression

```
Procedure Main is
   Result : Integer := Add (2 , 2);
   Value : Integer := 0;
begin
   if Add ( Value , 19) > 100
   then.....
```

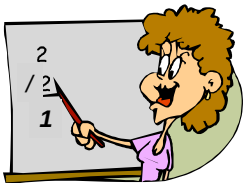
Parameters in brackets; separated by a comma

Must be an Integer

Can be object or literal

Defining Procedures

Procedures do not return a result:



Any local declarations

Designator

Parameter(s)

```
procedure Divide (X , Y : in Integer; Answer : out Integer) is
begin
    if Y = 0
    then
        return;
    else
        Answer := X / Y ;
    end if;
end Divide;
```

Answer undefined

Implementation

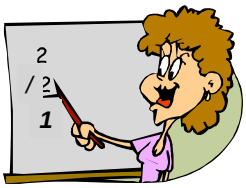
What does this *procedure* do?

Can contain a return statement, but this is NOT recommended

End of Divide

Calling Procedures

Procedure are called as standalone statements



- The result of calling a Procedure:
  - Cannot be assigned to an object
  - Cannot be used as part of an expression

Procedure call as standalone Ada statement

```
Procedure Main is
    Result : Integer ;
    Value : constant Integer := 2;
begin
    Divide (2 , Value , Result)
    if Result /= 0
    then…..
```

Parameters in brackets; separated by commas

Can be object or literal

Subprogram Parameters

Parameters are used to pass this information to subprograms

- Parameters can take one of three modes:

Considered a constant (inside the subprogram) initialised by the caller

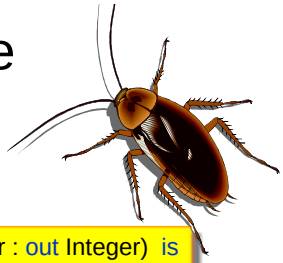
Is updated by the subprogram

```
procedure Do_Stuff  (Param_1 : in      Integer;
                     Param_2 : out     Boolean;
                     Param_3 : in out  Float)  is
begin
    ……..
```

Is a variable, initialised by the caller; it permits both reading and updating within the subprogram



An Exercise



What is wrong with the following:

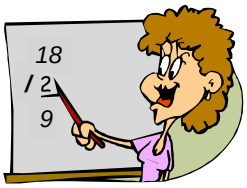
```
procedure Sum  (X , Y : in Integer ; Answer : out Integer)  is
begin
    Answer := X + Y ;
end Sum;
```

```
Procedure Main is
    Result : Float ;
    Value : constant Integer := 2;
begin
    Sum (1 , 2 , 3);
    Sum (2 , Value , 19);
    Sum (19 , Result , Value);
    Sum (2 , Value , Result);
```



Parameters Association

Parameters can be positional or named:




```
procedure Divide  (X , Y      : in Integer;
                   Answer     : out Integer)  is
begin
    .......
```

```
procedure Main is
    Object_1 : Integer := 2;
    Object_2 : Integer ;
begin
    Divide (18 , Object_1 , Object_2) ;
```

Positional Association → Divide (18 , Object_1 , Object_2) ; ← Less maintainable

Named Association "Better Practice" → Divide (Y => 18 , X => Object_1 , Answer => Object_2) ; ← Don't need to be in order

Default Parameters

In parameters can be given a default value:



- Actual values can be omitted from the call
  - If omitted, the default value is used
- If more than one parameter is defaulted:
  - Any number can be omitted from the call

```
procedure Display (Name      : in String ;
                   Forwards  : in Boolean := True );
```

Default value

```
Display (Name => "Objektum");
```

Uses default value for *Forwards*

```
Display (Name      => "Objektum",
         Forwards  => False);
```

Assigns value for *Forwards*

Subprogram Overloading

Functions and Procedures can be overloaded

- Two subprograms with the same name
  - The compiler can distinguish which subprogram is required by the types of the parameters passed



```
type Traffic_Light is (Red , Orange , Green);
type Apple is (Red , Green , Eaten );

procedure Print (Item : in Traffic_Light);
procedure Print (Item : in Apple);
```

```
Print (Item => Eaten);              ✓ Apple
Print (Item => Orange);             ✓ Traffic Light
Print (Item => Green);              ✗ Ambiguous
Print (Item => Apple'(Green));      ✓ Apple
```

Ambiguous calls must be qualified →

Subprogram Notes

- Subprograms are a good way of breaking up large programs into manageable chunks
- Functions:
  - Always return a value
  - Are called as components of expressions
  - Parameter mode must be *in*
- Procedures:
  - Are called as statements standing alone
  - Do not form part of an expression
- Parameters can be positional or named
  - Named parameters don't need to be in order



Defining Operators

It is possible to overload the predefined language operators:

- Its name must be a character string in double quotes
- The string must be one of the language operators
  - Defines a new meaning for the operator



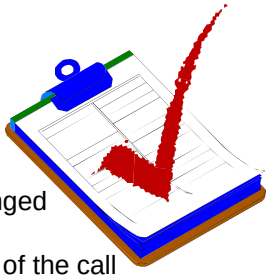
```
function "+" (Left , Right : in Array_Type) return Float is
      R : Float := 0.0;
begin
      for I in Array_Type'Range
      loop
            R := R + Left (I) + Right (I);
      end loop;

      return R;
end "+";
```

```
declare
      A_1 : Array_Type := (……);
      A_2 : Array_Type := (……);
      Sum : Float := A_1 + A_2;
```


Operator Notes

- Defining operators (or subprograms) with the same name is known as overloading
  - Overloading is resolved by the compiler
- All operators are simply a series of functions
- The precedence of the operators cannot be changed
- Overloaded operators cannot change the syntax of the call
- Must be one of the predefined language operators:



abs	and	mod	not	or	rem	xor
=	/=	<	<=	>	>=	
+	-	*	/	**	&	



Scope and Visibility



The scoping rules apply to subprograms:

```
procedure Outer is
      P : Integer;
      ...
      procedure Inner (Obj : Integer) is
            K : Integer := I ;
            I  : Integer := Obj;
            P : Integer := Outer.P ;
      begin
            K := Obj * I;
      end Inner;
begin
      ....
      Inner (Obj => P) ;
end Outer;
```

It is good practise to pass parameters to internal subprograms, even if the object is visible

Internal subprogram that is only visible within Outer

Inner can only be called from within Outer

Packages

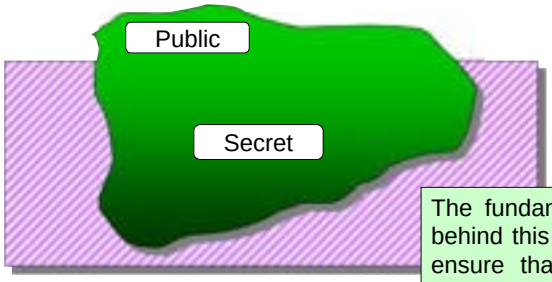
Objective



- To understand how a program is composed of separate modules (packages)
- To understand how to define Ada package specifications (interface) and bodies (implementation)
- To understand how to use packages
- To Understand how to split a large package into separate compliable units (separates)

Encapsulation

Encapsulation is the process of hiding the implementation from anything external to the object



The fundamental reason behind this principle is to ensure that changes to the private elements do not effect modules who use it, i.e. The *Clients*

Whatever is encapsulated can only be manipulated through public interfaces

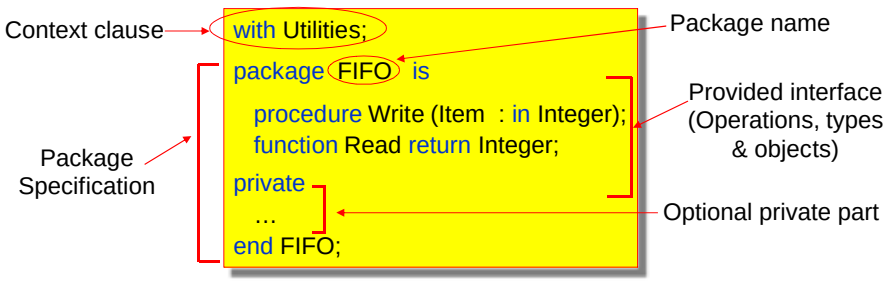
The Package *Specification*

Ada supports encapsulation through packages:



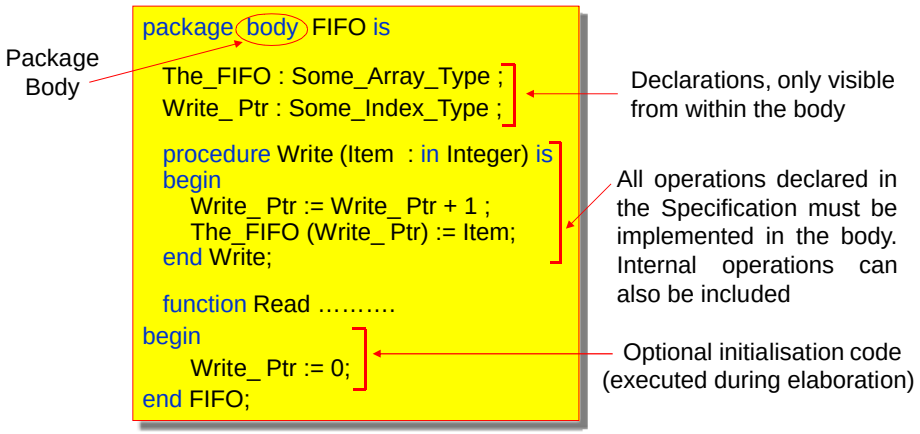
A package comes in two parts:

- Specification - Provides the visible Interface
- Body - Hidden implementation details



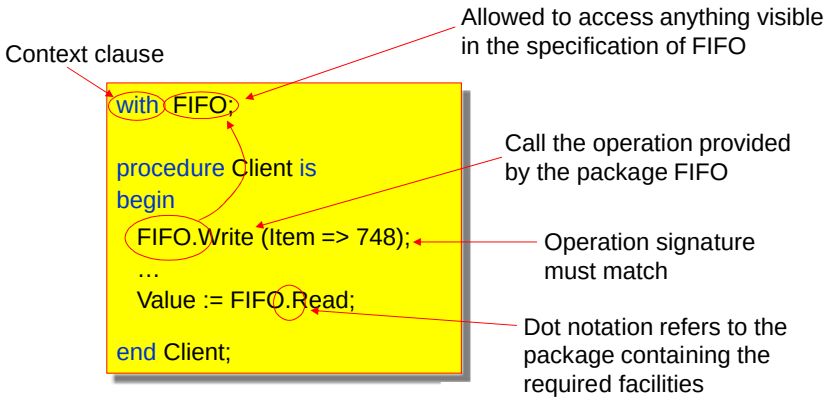
The Package *Body*

The implementation is hidden inside the package body:



The *with* Clause

Makes other packages (Library units) visible:



The *use* Clause

Makes entities within other packages visible without dot notation:

Context clause

Allowed to access anything visible in the specification of FIFO without dot notation

```
with  FIFO ;
use  FIFO ;

procedure Client is
begin
   Write (Item => 748);
   …
```

Call the operation provided by the package FIFO without dot notation



- The use clause is not recommended
  - Programs lose visibility of where entities originate
  - It is forbidden in most organisations

Declaring Separates

Allows the body of a package, or subprograms, to be taken-out of the embracing library unit and compiled separately:

```
package  body  FIFO is

   The_FIFO : Some_Array_Type ;
   Write_ Ptr : Some_Index_Type ;

   procedure Write (Item  : in Integer) is separate;

   function Read return Integer is separate;

end FIFO;
```

Normal package level declarations

Actual implementation is in a separate file



The removed units are termed subunits and can be compiled separately.

Implementing Separates

Must be preceded by separate and the name of the parent unit:

Must start with separate — Parent name — No semicolon

```
separate (FIFO)
procedure Write (Item : in Integer) is
begin
      ……...
end Write;
```

Implementation details now partitioned into a separately compilable unit

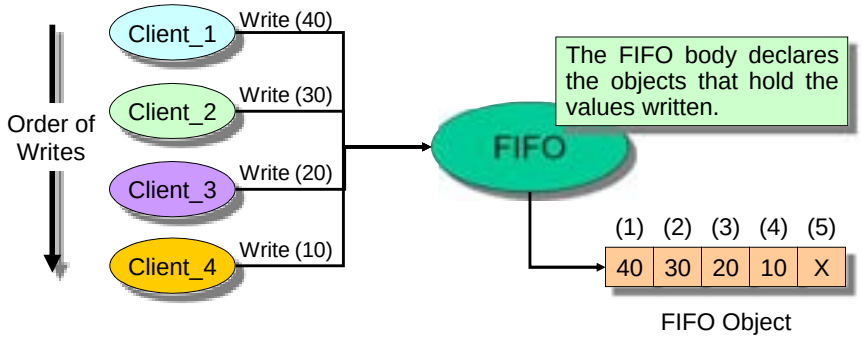
The parent itself could be a subunit:

```
separate (Parent.Subunit_1.Subunit_2)
```

Subunit_2 is a subunit of Subunit_1 which is a subunit of Parent

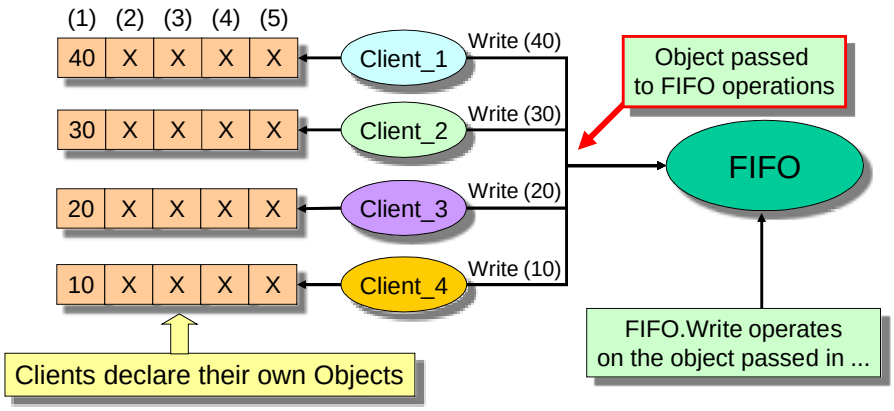
Encapsulation Problems

Our FIFO package encapsulates the data within the package body. This means that we only have 1 FIFO in our system:



Encapsulation Problems

We could declare a type (to represent the FIFO) in the specification:



Exporting Types

The corresponding specification would be:

```
package FIFO is

    type FIFO_Type is
    record
        The_FIFO : Some_Array_Type ;
        Write_ Ptr : Some_Index_Type ;
    end record;

    procedure Write (Item : in Integer ;
                     This : in out FIFO_Type);

    procedure Read (Item : out Integer ;
                    This : in out FIFO_Type);
end FIFO;
```

Users can declare their own FIFO objects

FIFO object passed to Write & Read operations

Mode is 'in out' because Write & Read modify the object

Cannot be a function because Read modifies the object

Using Exported Types

Clients must now declare their own FIFO objects:

Context clause

```
with FIFO;
procedure Client is
   My_FIFO : FIFO.FIFO_Type;
begin
   FIFO.Write (Item => 748 ,
               This => My_FIFO);

end Client;
```

Client now owns the FIFO object

Object passed to Write which knows how to operate on an object of type FIFO

- Because the structure of the FIFO is 'public', clients could interfere with the components:

```
My_FIFO. Write_ Ptr := 0;
```

Will work, should be illegal

Intentionally Left Blank

Private Types

Objective



- To understand how a package can export types
- To understand how to export types but keep the implementation hidden using ‘private’
- To understand what private and limited private types are and what operators are available
- To Understand what derived types are

The Package *Specification*

Ada supports encapsulation through packages:



A package comes in two parts:

- Specification - Provides the visible Interface
- Body - Hidden implementation details

Context clause

```
with Utilities;
package FIFO is
   procedure Write (Item  : in Integer);
   function Read return Integer;
private
   ...
end FIFO;
```

Package name

Provided interface (Operations, types & objects)

Package Specification

Optional private part

Exporting Types

The corresponding specification would be:

```
package FIFO is

   type FIFO_Type is
   record
      The_FIFO : Some_Array_Type ;
      Write_ Ptr : Some_Index_Type ;
   end record;

   procedure Write (Item : in Integer ;
                    This : in out FIFO_Type);

   procedure Read (Item : out Integer ;
                   This : in out FIFO_Type);
end FIFO;
```

Users can declare their own FIFO objects

FIFO object passed to Write & Read operations

Mode is 'in out' because Write & Read modify the object

Cannot be a function because Read modifies the object

The *private* Part

Enables us to create Abstract Data Types:

FIFO declares a type, but hides its representation

Procedure declarations remain unchanged

Visible part

Private part

```
package FIFO is
    type FIFO_Type is private;
    procedure Write ……
    procedure Read …….
private
    type FIFO_Type is
    ……
end FIFO;
```



"Clients cannot see the private part"

Full type definition is declared in the private part of the specification

```
My_FIFO. Write_ Ptr := 0;
```

Is now illegal

Private Part Notes

- The private part is in the specification because:
  - The representation might change and therefore any objects declared of the private type must be recompiled
- It can be guaranteed that:
  - If the representation of a private type is changed, the rest of the program will not be affected
- Private types restricts the operators to:
  - Assignment
  - Equality Comparison
  - Those defined in the package specification



Private Type Operators

Are automatically restricted to assignment and equality comparison:

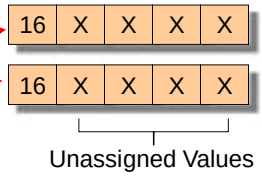
The comparison operator provided might be inappropriate, consider:

```
with FIFO;
procedure Client is

   FIFO_1, FIFO_2 : FIFO.FIFO_Type;

begin
   FIFO.Write (Item => 16 , Object => FIFO_1);
   FIFO.Write (Item => 16 , Object => FIFO_2);

   if FIFO_1 = FIFO_2
   then
        ……
```



Would unnecessarily test the entire contents of each FIFO object

Private Type Operators

Can be defined so we can provide our own implementation:

```
Package FIFO is

   type FIFO_Type is private;

     procedure Write…..
     procedure Read …..

     function "=" (Left, Right : FIFO_Type) return Boolean;
     …
end FIFO;
```

FIFO can now decide what the equality function does

We can ensure that 'True' is only returned if the FIFOs have the same number of values written and all the 'written' values are identical

Will be implemented in the body

Operator Visibility Rules

- If a type is declared in a package specification:
  - Operators are not directly visible
  - Even if a *with* clause is used
- The *use* clause gives direct visibility of all declarations that appear in the visible part of a package specification:
  - Includes all the operators +, -, /, = etc.
- Most companies outlaw the *use* clauses completely:
  - Obscures the origin of entities



Making Operators Visible

A rename for the required operator must be used:



```
with FIFO;
procedure Client is

   FIFO_1 , FIFO_2 : FIFO.FIFO_Type;

   function "=" (L,R: FIFO.FIFO_Type)
      return Boolean renames FIFO."=" ;
begin
      ……

      if FIFO_1 = FIFO_2 ...
```

No *use* clause for FIFO; therefore "=" is not visible and must be renamed

Operator rename

When "=" is required, the one provided by FIFO is used

Limited Private Types

Restrict the operators to those declared in the specification:

```
Package FIFO is
   type FIFO_Type is limited private;
      procedure Write…..
      procedure Read …..
Private
      type FIFO_Type is …
end FIFO ;
```

Type is now *limited private*, hence, No operators are automatically available

Objects of FIFO_Type cannot be copied or checked for equality

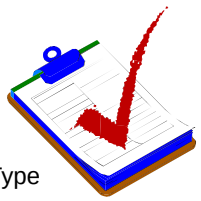


The advantage of making types limited private is to allow the package writer to have complete control over the objects of the type, i.e. assignment & equality

Derived Types

Are used to introduce a new type which follows the characteristics of an existing type but are distinct:

```
type New_Type is new Existing_Type;
```



- New_Type is said to be a derived type and Existing_Type
- New_Type belongs to the same class as Existing_Type
- The set of values in New_Type are the same as Existing_Type
- Primitive operators are inherited from Existing_Type
- For a type declared in a package specification, subprograms with a parameter or result of the type are also automatically inherited

Derived Types

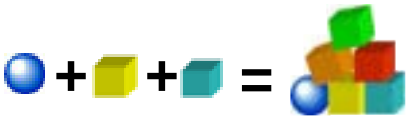
Although derived types are distinct, due to their relationship with their parent, they can be directly converted:

```
N : New_Type
E : Existing_Type;
N := New_Type (E);
```

type cast

E cannot be directly assigned to N as they are different types

Why use derived types ?



Don't want it to Compile

Avoids the accidental mixing of objects of conceptually different types

Derived Types

Are in some ways an alternative to private types and have the advantage of inheriting literals:

```
type Length_Type is new Float;
type Area_Type is new Float;
```

Two distinct types derived from Float

- However they have the disadvantage of inheriting too much:
  - Prevents the mixing of lengths and areas but inherits the ability to multiply two lengths giving a length as well as other irrelevant operators

```
Length_1 := Length_2 * Length_3;
```

Valid Ada, but dimensionally incorrect

Inherited from Float



Derived Types

Area and length should be private:

```
package Maths_Stuff is

   type Area_Type is private;
   type Length_Type is private;

   function "+" (Left, Right : Length_Type) return Length_Type;
   function "*"  (Left, Right : Length_Type) return Area_Type;

private

   type Area_Type is new Float;
   type Length_Type is new Float;

end Maths_Stuff;
```

Only assignment and equality are automatically available

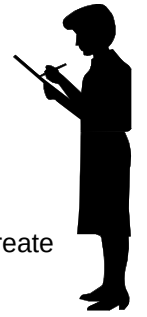
Defined set of operators



Two lengths can be added, but if they are multiplied, the result will always be an area.

Hierarchical Libraries

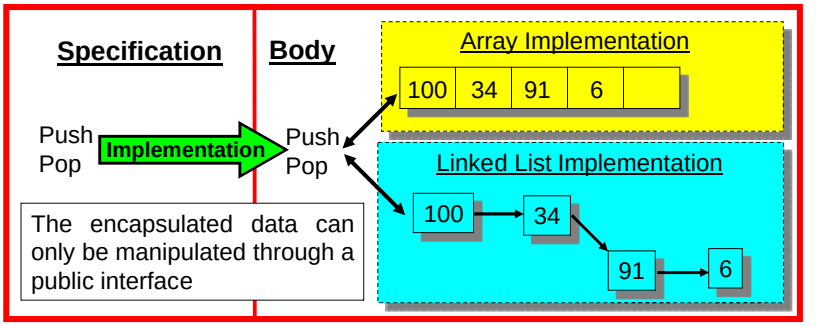
Objective



- To understand the new rules for packages introduced by Ada95
- To understand how a program can be better decomposed using child packages and how to create them
- To understand the visibility rules that apply to child packages
- To understand how the predefined language environment is structured using child packages

Concept of Packages

One of the strengths of Ada is the package where the distinct specification and body decouple the interface and implementation



A Stack Package (with two different implementations)

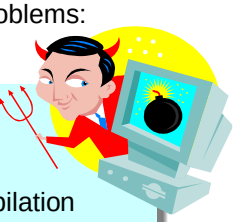
New Package Body Rules

- Ada83 allows bodies to be defined for packages without subprograms
  - This caused serious problems when bodies which contained just an initialisation part got inadvertently missed during system build
- Ada95 only allows a library package to contain a body if it is required by language rules:
  - The package has subprograms
  - pragma Elaborate_Body is included in the specification



Ada83 Package Problems

However, in Ada83 this did cause some irritating problems:



- Packages become large, monolithic units
- Impossible to extend a package without recompilation
- If several private types need access to each other's implementations, they all had to be enclosed together in the same package
- Potential name clashes with a flat library structure

Child Packages





- Hierarchical library structures containing child packages and subprograms
- Children can be added to extend the functionality of the parent package without extensive recompilation
- Children can be
  - Public
  - Private
- A child package is an independent library unit and is not visible unless referenced by a with clause

Naming Child Packages

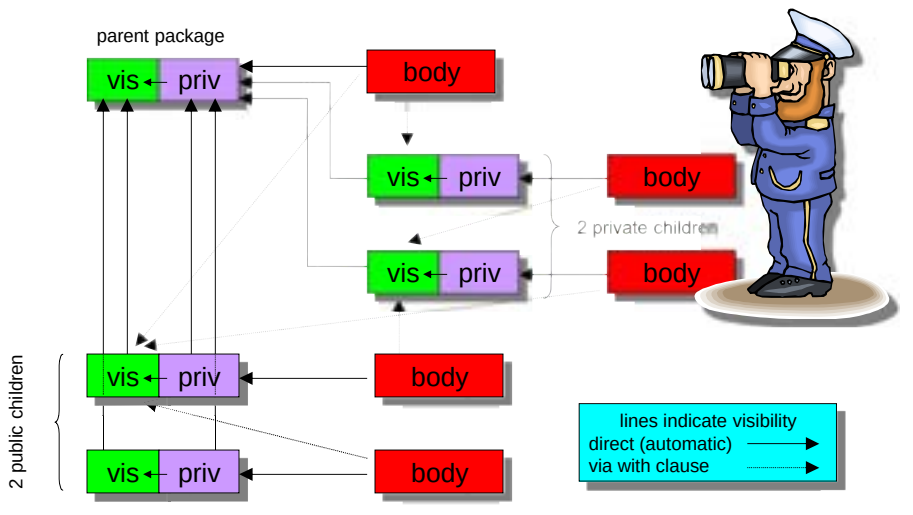
```
package Parent is
      type Some_Type is private ;
      procedure Do_Something ...
private
      type Some_Type is .....
end Parent;
```




```
package Parent.Child_1 is
      procedure Do_More.....
end Parent.Child_1 ;
```

```
private package Parent.Child_2 is
      procedure Do_Secret.....
end Parent.Child_2 ;
```

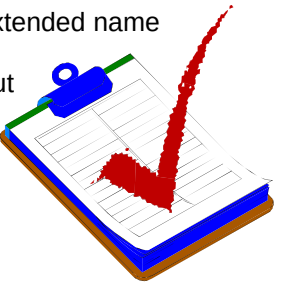

Visibility



Summary

The following list summarises the changes introduced by Ada95:

- Child packages are separately compiled library units and can be used to extend a package
- Child packages can be public, private
- Child packages are named using an extended name
- Private children can be modified without extensive recompilation

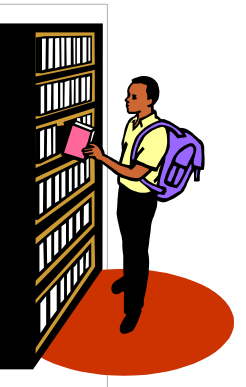


Predefined Language Environment

The main objective of Ada95 was to supply a set of supplemental packages (of general purpose utilities) in order to promote portability and reuse.

This was achieved through the use of child packages :

- Ada
- Interfaces
- System



Will be covered in Predefined Library section

Intentionally Left Blank

Ada83
Generic Programming

Objective

- To understand the concept of generic programming
- To understand how to define generic sub programs and packages and how to instantiate them
- To understand how types, objects and sub programs can be used to parameterise generic sub programs and packages



The Concept

In order to swap two items, it is not necessary to do any operations other than the assignment of one variable to another:

```
procedure Swap (A : in out Integer;
                B : in out Integer) is
    Temp : constant Integer := A ;
begin
    A  := B;
    B  := Temp;
end Swap;
```

Only works with integers

To swap variables of different types, we would have to 'cut & paste' the procedure and only change the word Integer



Generic Subprograms

Can be modified to make generic :

Formal Parameter

```
generic
    type Item_Type is private;
procedure Generic_Swap (A : in out Item_Type;
                        B : in out Item_Type);
```

Specification

```
procedure Generic_Swap (A : in out Item_Type;
                        B : in out Item_Type) is

    Temp : constant Item_Type := A;
begin
    A := B;
    B := Temp;
end Generic_Swap;
```

Body

Generic Subprograms

To create two procedures to swap Integers & Floats:

type to use for 'Item_Type'

Positional association

```
procedure Swap_Integers is new Generic_Swap (Integer);
procedure Swap_Floats is new Generic_Swap (Item_Type => Float);
```

Named association

This process is known as Generic Instantiation

The procedure can now be called in the normal way:

```
Swap_Integers (A => Integer_1, B => Integer_2);
Swap_Floats (A => Float_1, B => Float_2);
```


Generic Packages

Generics can take the form of packages too:

```
generic
      type Item_Type is private;
package Generic_Stack is
      procedure Push (Value : in Item_Type);
      function Pop return Item_Type;
end Generic_Stack;
```

Formal Parameter

Specification

```
package body Generic_Stack is
      procedure Push (Value : in Item_Type) is ...
      function Pop return Item_Type is ...
end Generic_Stack;
```

Body

(details omitted)

Generic Packages

Generic packages must also be instantiated before use

```
package My_Stack is new Generic_Stack (Item_Type => Integer);
```

Stack of integers

If instantiated at library level, the generic Instantiation may be 'withed' as a normal package

```
with My_Stack;
procedure Main is
   Popped_Value : Integer;
begin
   My_Stack.Push (Value => 916);
   Popped_Value := My_Stack.Pop;
end Main;
```

Operations called in the normal way

Object Parameters

Objects can be passed as formal parameters:

Not considered to be static within the generic

```
generic
   Size_Of_Stack : in Integer;
package Generic_Stack is …
```

'in' Objects act as constants within the generic

```
generic
   Number_Of_Items : in out Integer;
package Generic_Stack is …
```

'in out' Objects act as variables within the generic

There is no concept of 'out' parameters. If the mode is omitted, it defaults to 'in'

Type Parameters (Private)

Type parameters specify types (and associated operations) that may be used within the generic.

Formal type parameters of type 'private' may be used to specify types where only assignment and equality are known to be available

type T is private; → T may be of any non-limited type. Variables of this type may be declared within the generic.



Only assignment and equality functions are immediately available within the **body** of the generic

Type Parameters (Limited Private)

Formal generic parameters can also be specified as limited private:

type T is limited private; → T may be of any type. Variables of this type may be declared within the generic



With limited private type parameters, NO assumptions can be made about the instantiated type

Either of the previous forms could have discriminants:

type T (…) is private; → T must have discriminants with the same type.

Type Parameters (Scalar)

Formal generic parameters can be any scalar type:

```
type T is (<>);
type T is range <>;
type T is digits <>;
type T is delta <>;
```

- `type T is (<>);` → T is any discrete type, either an enumeration or integer type
- `type T is range <>;` → T is any signed integer type
- `type T is digits <>;` → T is any floating point type
- `type T is delta <>;` → T is any fixed point type



When this form is used, the generic body has available to it all of the operations and attributes that are normally available for these types

Type Parameters (Array)

Formal generic parameters can be specified to be arrays:

In this case it is necessary for the compiler to know what type the array is indexed by, and what type each element of the array is:

```
type T is array (I range <>) of S;
```

→ T is an array of S, indexed by type I (where S and I may also be generic formal parameters)

For Example:

```
generic
   type S is limited private;
   type I is (<>);
   type T is array (I range <>) of S;
```

→ S may be any type, I must be a discrete type (enumeration or integer) and T is array of S indexed by I

Type Parameters (Access)

Formal generic parameters can be specified to be access types:

```
type A is access T;
```

A must be of type access T.

- When types are passed as formal parameters, the body of the generic can use the appropriate:
  - Predefined operators
  - Attributes
- Sometimes, relying on the predefined operators is not sufficient



Subprogram Parameters

Formal generic parameters can be specified to be access types:

```
generic
   type Item_Type is (<>);
package Generic_Stack is
   procedure Push (Value : in Item_Type);
   function Pop return Item_Type;
   procedure Sort;
end Generic_Stack;
```

Any discrete type, so the body can rely on using comparison operators

Stack now couldn't store records, floats etc.

```
package My_Stack is new Generic_Stack (Item_Type => Float);
```

Not Discrete

Subprogram Parameters

Comparison operator can be passed as a parameter:

Within the body of the generic, "<" can be called

Can now be float, record etc.

```
generic
    type Item_Type is private;
    with function "<" (Left, Right : Item_Type)
        return Boolean;
package Generic_Stack is
    ……
```

Must be visible and match the required spec.

```
package My_Stack is new Generic_Stack
    (Item_Type  => Float,
     "<"        => "<")
```

In this case, "<" for float is always visible

Subprogram Parameters

Default operations can be specified for subprogram parameters:

```
with function "<" (Left, Right : Item)
  return Boolean is <>;
```

Specifies that when the instantiation takes place, a subprogram that is visible will be used (if it matches the specification required).



This is particularly useful when using subprogram specifications that match predefined operations

```
package My_Stack is new Generic_Stack
    (Item_Type  => Float);
```

Predefined type, therefore "<" will always be visible

Subprogram Parameters

The second form of default parameter (less commonly used) :

```
with function "<" (Left, Right : Item)
   return Boolean is Less_Than;
```

Specifies that the default subprogram is the one that is visible at the time of the generic declaration

Since any parameters that are required to be of the generic formal type are not known at this point, it is only of use for subprograms that do not have parameters of the generic formal type

Intentionally Left Blank

Ada95
Generic Programming

Objective

- To understand the new generic formal parameter notation that has been added to Ada95
- To understand how packages can be passed to generics using Ada95
- To understand how child packages must be structured when using generics



Scalar Type Parameters

Two additional forms of Scalar generic parameters have been added:

`type T is mod <>;` → T is any modular type

`type T is delta <> digits <>;` → T is any decimal type



- These additions accommodate the new Ada95 scalar types
- Once again, all of the operations and attributes that are normally available for modular and decimal types are available in the body of the generic

Access Type Parameters

Additional forms of Access generic parameters have been added to accommodate general access types:

`type A is access all T;` → A must be a general access to variables of type T.

`type T is access procedure…;` → T must access procedure as specified

`type T is access function…;` → T must access function as specified



The profiles of the actual and formal subprograms must match. i.e. the number of parameters and their associated mode.

Indefinite Type Parameters

Ada83 did not distinguish between indefinite and definite parameters:

```
generic
      type T is private;
procedure Swap (A,B: in out T);
procedure Swap (A,B: in out T) is
      Temp : T := A ;
```

T may be of any non-limited type. Variables of this type may be declared within the generic body.

Results in a violation of the contract model...

```
with Swap;
procedure Main is
     procedure My_Swap is new
          Swap (T => String);
begin ……
```

It Will NOT Instantiate

Indefinite Type Parameters

New for Ada95 is the indefinite type generic parameter:

NEW!

```
type T (<>) is private;
```

T may be of any definite or indefinite type thereby prohibiting the declaration of uninitialised variables in the body

Generic formal parameters can also be specified as limited private:

```
type T(<>) is limited private;
```

T may be of any definite or indefinite type. Obviously, assignment and equality are not available in the body of the generic

Tagged Type Parameters

Another addition is the inclusion of tagged generic parameters:

type T is tagged private; → T must be tagged, but not abstract

type T is abstract tagged private; → T must be tagged, may be abstract

This example shows the above forms put together:

type T (I : Index) is abstract tagged limited private;

T must have 1 discriminant, must be tagged, may be abstract, may be limited

Derived Type Parameters



Generic formal parameters can now be derived types.

type T is new S; → T must be derived from S, where S is not a tagged type.

type T is new S with private; → T must be derived from S, where S is a tagged type.

- The formal parameter has to be instantiated with a type that is derived from another specified type
- The body of the generic may assume that all of the properties of the parent type are available

Package Parameters

Ada95 allows packages to be passed as formal parameters

- Allowing a series of consistent instantiations to be created, with each generic depending on another:

```
generic
   with package Utilities is new Generic_Utilities (<>);
package Queue…
```

- The above notation (using <>) specifies that:

  When Queue is instantiated, a package must be provided as an actual parameter, and that package must itself be an instantiation of the specified generic package Generic_Utilities

Package Parameters

An example:

Generic_Procedure must be instantiated with an actual parameter that is an instantiation of Integer_IO:

```
generic
      with package Display is new Ada.Text_IO.Integer_IO (<>);
procedure Generic_Procedure (Item : in Display.Num);
```

```
with Ada.Text_IO;
with Generic_Procedure;
….
      package Int_IO is new Ada.Text_IO.Integer_IO (Integer);
      procedure My_Procedure is new
            Generic_Procedure (Display => Int_IO);
```

First instantiation passed into the second

Package Parameters

Formal parameters specify explicit actual parameter types :

```
type Byte_Type is range 0..255;

generic
   with package Display is new Ada.Text_IO.Integer_IO (Byte_Type);
procedure Generic_Procedure (Item : in Byte_Type);
```

- Generic_Procedure must be instantiated with an instance of Integer_IO
- Integer_IO must have been instantiated with Byte_Type

```
package Byte_IO is new
   Ada.Text_IO.Integer_IO (Byte_Type);
procedure My_Procedure is new
   Generic_Procedure (Display => Byte_IO);
```


Generic Child Packages

Child packages can be Generic:

- If the parent is not generic then a generic child may be instantiated in the normal way (at any point it is visible)

```
package Parent is
   …
end Parent;
```

```
generic
   type P is private;
package Parent.Child is
   …
   …
end Parent.Child;
```

```
with Parent.Child;
package User is
   package Child_Instance is new
      Parent.Child (P => Who_Cares);
```



Generic Child Packages

Generic parent example

- If a parent is generic then all its children must be generic and the rules of instantiation are different

```
generic
   type P is private;
package Parent is
   …
end Parent;
```

```
generic
package Parent.Child is
   …
end Parent.Child;
```

Note: No use clause for the parent, as the one for the child implies one

```
with Parent.Child;
package User is
   package Parent_Instance is new
      Parent (P => Who_Cares);
   package Child_Instance is new
      Parent_Instance.Child;
```

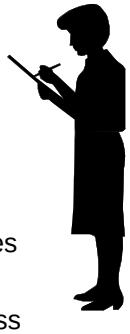


The instantiation of the child refers to the parent instance

Intentionally Left Blank

Ada83 Access Types

Objective



- To understand the concept of pointers in Ada83 through the use of Access types and dynamic memory allocation
- To understand how to define Ada83 access types
- To understand how to declare objects and access them indirectly through Ada83 access types
- To understand how a simple linked list can be implemented using Ada83 access types and how a ragged array could be created.

The Concept

Access types are the Ada name, more commonly known in other languages as pointers

A pointer is a variable that is capable of holding the address of another variable

- All variables have a unique address



Pointers have been an important feature of most languages but provides raw flexibility that is open to abuse and is often the cause of many erroneous programs.

Declaring Access Types

An access type is declared as a pointer to another type



Variable that can point at integers

```
type Ptr_Type is access Integer ;
Ptr : Ptr_Type ;
begin
Ptr := new Integer ;

Ptr := new Integer ‘ (999) ;
```

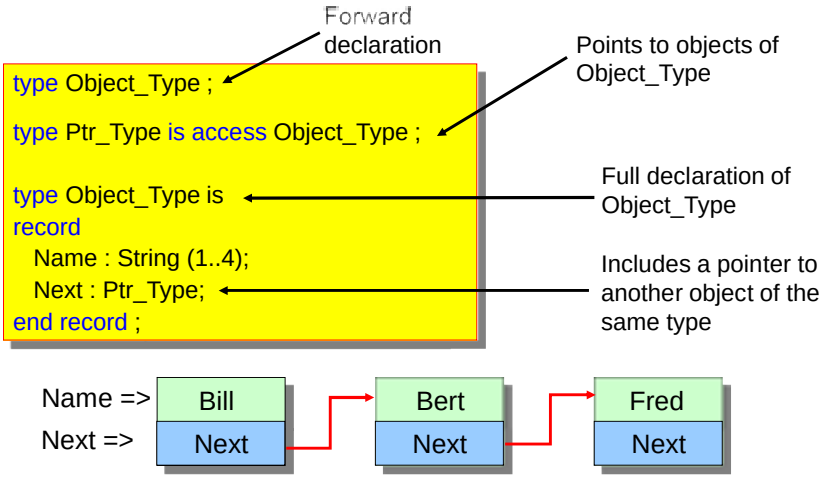
Points to null

Creates an object of type integer

Creates an object of type integer with an initial value of 999

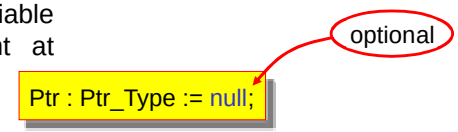
Declaring Access Types

One of the simplest uses of an access type is for list processing:

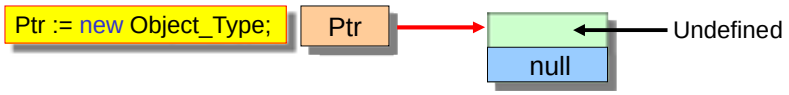


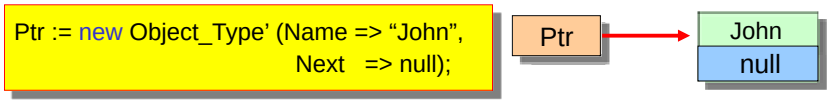
Creating Objects

Declares a variable that can point at object_type



Objects are created by the execution of an allocator (which optionally can provide a default value):

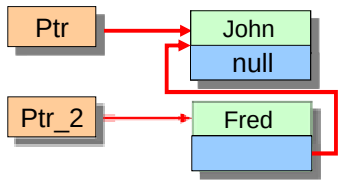




Creating Objects

To create a further record, we could write:

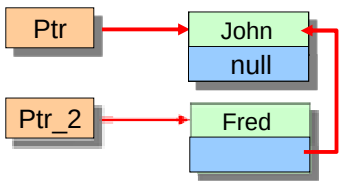
```
Ptr_2 : Ptr_Type ;
Ptr_2 := new Object_Type' (Name => "Fred" ,
                           Next  => Ptr);
```



List_2.Next has been set to the same value as List, which points at John's object

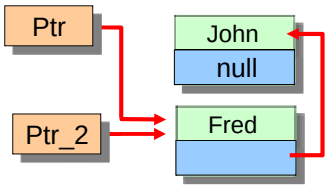
Accessing Objects

Assuming we started with:



To copy the pointers, we could write:

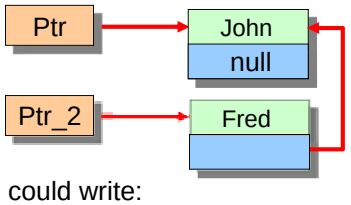
```
Ptr := Ptr_2;
```



The pointers are copied, **NOT THE OBJECTS**

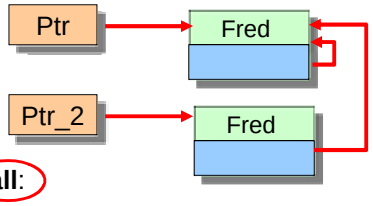
Accessing Objects

Assuming we started with:



To access or copy the values, we could write:

```
Ptr.Name := Ptr_2.Name;
Ptr.Next   := Ptr_2.Next;
```



Alternatively,
we could use the reserved word **all**:

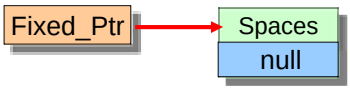
```
Ptr.all := Ptr_2.all;
```

.all refers to the whole object

Accessing Constants

A constant access type must be provided with an initial value:

```
Fixed_Ptr : constant Ptr_Type :=
      new Object_Type' (Name => "    " ,
                        Next   => null );
```



Fixed_Ptr is constant and therefore always points at the same object but the value of the object itself can be changed:

```
Fixed_Ptr.all := Ptr.all ;

Fixed_Ptr := Ptr ;
```

attempt to write to a constant

Summary

```
if Ptr = Ptr_2
if Ptr.all = Ptr_2.all
```

True if the pointers point at the same object

True if the object values are the same

Consider the following:

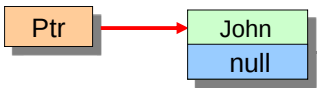
```
type Ptr_1_Type is access Integer;
type Ptr_2_Type is access Integer;
Ptr_1 : Ptr_1_Type;
Ptr_2 : Ptr_2_Type;
….
Ptr_1 := Ptr_2 ;
Ptr_1.all := Ptr_2.all ;
```

Will not compile, different types

Is Ok, both point at the same type of object

Adding Elements to a List

Assuming we started with:

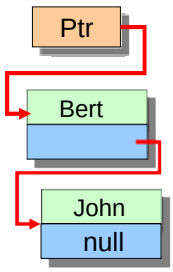



```
procedure Add_To_List (Ptr : in out Ptr_Type ;
                       Name  : in String) is
begin
  Ptr := new Object_Type' (Name => Name ,
                           Next  => Ptr);
end Add_To_List;
```

…….

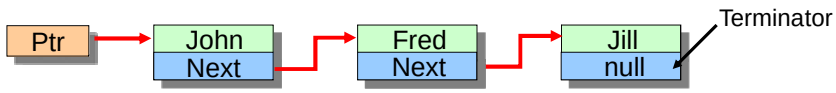
```
Add_To_List (Ptr => Ptr , Name => "Bert");
```

Would yield:



List Terminators

Consider a list containing personnel and we are required to count the number of people in the list.




```
function Count (Ptr : in Ptr_Type) return integer is
    Local_Ptr : Ptr_Type := Ptr;
    Count : Integer := 0;
begin
  while Local_Ptr /= null loop
    Count := Integer'Succ (Count);
    Local_Ptr := Local_Ptr.Next;
  end loop;
  return Count;
end Count ;
```

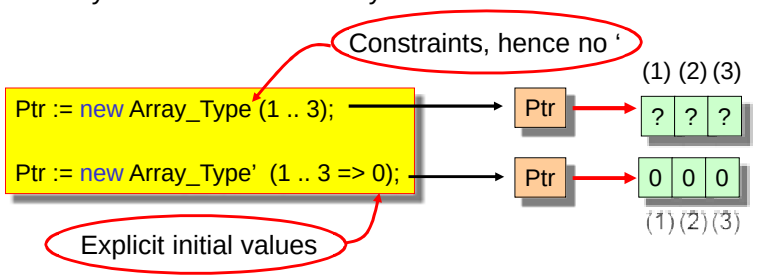
Read next pointer value

Access to Unconstrained Types

Access types can point to constrained and unconstrained arrays:

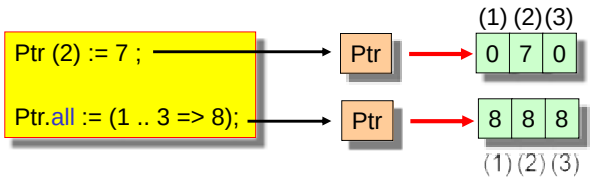
```
type Array_Type is array (Natural range <>) of Integer;
type Ptr_Type is access Array_Type;
Ptr : Ptr_Type ;
```

A new array can then be defined by either :



Access to Unconstrained Types

The components of an unconstrained object can be accessed in the usual way :



Because Array_Type is unconstrained, we could not write:



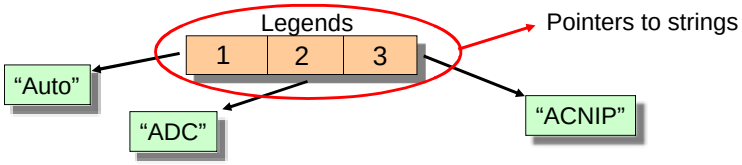
Creating a Ragged Array

```
type String_Ptr_Type is access String;

function "+" (S : String) return String_Ptr_Type is
begin
   return new String'(S);
end "+";

type Legend_Type is array (1..3) of String_Ptr_Type;
Legends : constant Legend_Type := (+"Auto" , +"ADC" , +"ACNIP");
```

Create object, assign it, return handle



Ada95 Access Types

Objective



- To understand the concept of pointers in Ada95 through the use of Access types
- To understand how to define Ada95 access types and what the accessibility rules are
- To understand how to declare objects and access them indirectly through Ada95 access types
- To understand how to create access types that point at procedures and functions
- To understand what access parameters are

The Concept

Ada83 access types provides a means of manipulating objects created by allocators

Ada95 includes general access types which can be employed to provide indirect access to statically declared objects







Aliased Objects



To allow an object to be referred to (or pointed at) it must be marked as aliased:

New for Ada95

```
Object : aliased Integer := 888;
```

Creates → Object: 888

An access type must then be declared to allow statically declared objects to be pointed at:

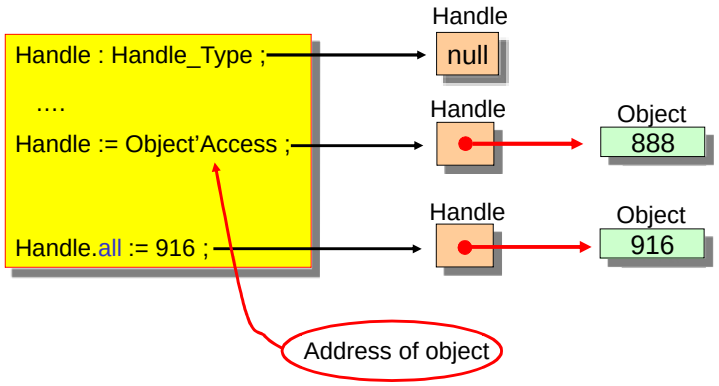
```
type Handle_Type is access all Integer;
```

New access notation for Ada95



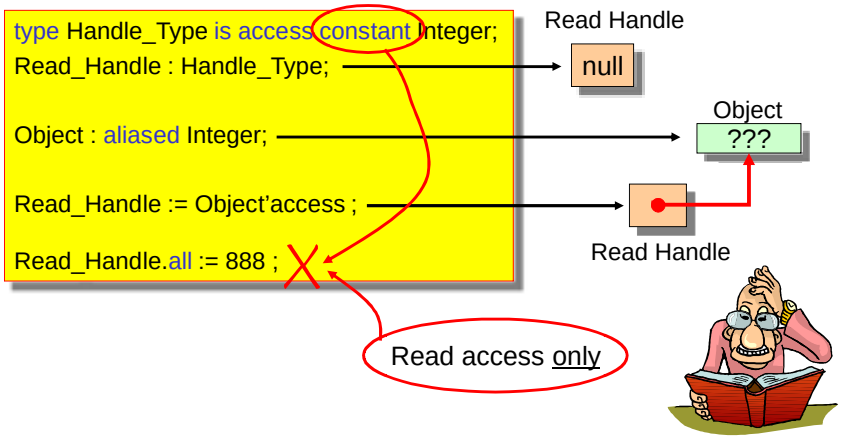
Accessing Objects

An access object can now be declared and assigned a value to reference the static object

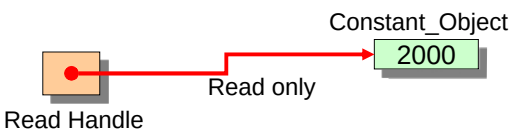


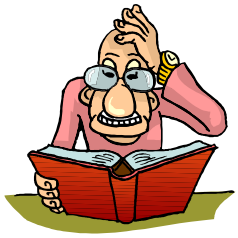
Read Only Access Types

A restriction can be imposed to ensure that access is read only:



Accessing Constants





```
Constant_Object : aliased constant Integer := 2000;

Read_Handle := Constant_Object'access ;

Read_Handle.all := 916 ;
```

Cannot write to a constant

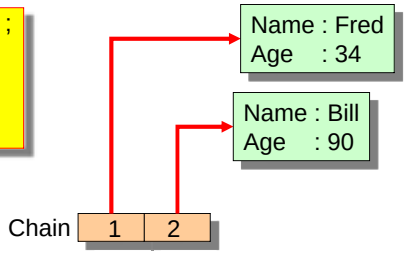
Chains of Static Data

Chains of statically declared objects to be created, consider:

```
type Handle_Type is access all Record_Type;
type Chain_Type is array (Natural range <>) of Handle_Type;
```

```
Fred, Bill : aliased Record_Type ;

Chain : constant Chain_Type :=
          ( 1=> Fred'Access,
            2 => Bill'Access);
```



Access to Subprograms

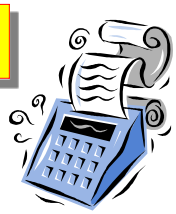


Ada95 includes the ability for access types to refer to subprograms:

```
type Trig_Function is access function (Data  :  in  Float)  return Float;
Operation : Trig_function ;
```

Points to Cos

```
function Cos (X : Float) return Float  is ….
function Sin  (X : Float) return Float is ….
```



```
Operation :=  Cos'Access ;
Result : Float := Operation (Data => Theta);
```

Indirect call

Dynamic Selection



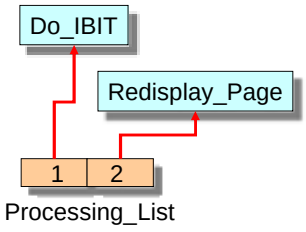
```
type Key_Type is range 1..20;
type Processing_Type is access procedure;
type List_Type is array (Key_Type) of Processing_Type;
```

```
procedure Do_IBIT ;
procedure Redisplay_Page;
…….
```

Matches signature

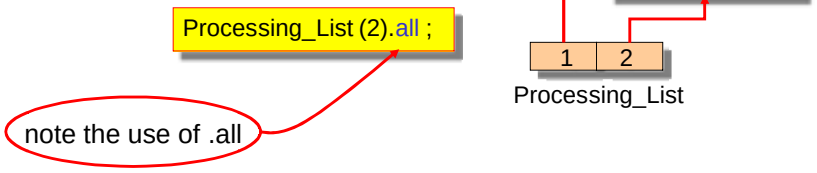
Create list of procedures:

```
Processing_List : List_Type :=
    (1 => Do_IBIT'Access ,
     2 => Redisplay_Page'Access ……..
```



Dynamic Selection

To call the attached operation, the appropriate access object is simply dereferenced:



If the operation has parameters, then **.all** is not required

Access Parameters



New for Ada95

```
procedure Display (Pointer : access Integer) is ...
```

Must be aliased

```
Object : aliased Integer := 995;
Display (Pointer => Object'Access);
```



Access parameters can never have a null value which means within the subprogram body, there is no need to check for null values before dereferencing

Accessibility Rules

Designed to ensure accessed objects can't go out of scope

- The basic rule is that the lifetime of an accessed object must be at least as long as the access type:

type has a longer life than objects in declare block

Will go out of scope

Points nowhere

```
procedure Accessibility is
    type Handle_Type is access all Integer;
    Handle_1 : Handle_Type;
begin
    declare
        Handle_2 : Handle_Type;
        Object : aliased Integer;
    begin
        Handle_2 := Object'Access;

        Handle_1 := Handle_2;
    end;
end Accessibility;
```

Will not compile

Named Access Discriminants

Ada95 allows a record discriminant to be of an access type

- Enables a record to be parameterised with subprograms or other data structures with which it is associated

```
type Record_Type (Object : Access_Type) is
record
    Assigned : Boolean := False;
    ……..
end record;
```

Named access type

Anonymous Access Discriminants

Can only be applied to records marked as **limited**

```
type Object_Type is …..

type Record_Type (Object : access Object_Type) is limited
record
   …
end record;
```

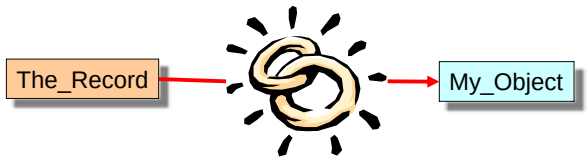
- Because the record type must be marked as limited, assignment is prohibited and therefore access discriminants are **always** considered to be constant.
- This means that once two objects are bound together, they are done so permanently, i.e. the access discriminant can never change.

Anonymous Access Discriminants

Declaration must include

- An access value to an associated object of type Object_Type:

```
My_Object :  Object_Type ;
The_Record : Record_Type (Object => My_Object'Access);
```



Bound together forever

Self-Referential Structure

A component of a record can obtain the identity of the record in which it is embedded:

```
type Vehicle_Type;

type Engine_Type (Vehicle : access Vehicle_Type) is limited
record
      Engine_No : Integer range 000..999;
end record ;

type Vehicle_Type is
record
      Reg_No  :  String (1..8);
      Engine   :  Engine_Type (Vehicle_Type'Access);
end record;
```

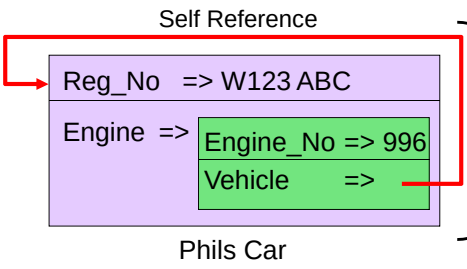
Access Discriminant refers to the type in which it is embedded

Self-Referential Structure

If a vehicle was to be declared and assigned:

```
Phils_Car : Vehicle_Type;
...
Phils_Car.Reg_No := "W123 ABC";
Phils_Car.Engine.Engine_No := 996;
```





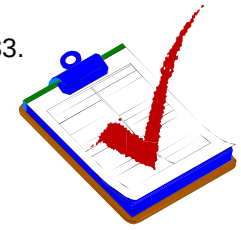
The Engine of type Engine_Type has an access discriminant (Vehicle) which refers back to the instance of record type Vehicle.

This is due to the attribute Access applied to the name of the record type inside its declaration.

Summary of Changes

The following list summarises the changes introduced by Ada95:

- Ada83 did not have general access types, only pool specific access types.
- Access to subprogram types did not exist in Ada83.
- Access parameters did not exist in Ada83.
- Discriminants could not be of an access type in Ada83.



Object Oriented Programming

Objective



- To understand the fundamental concept of Object Oriented Programming (objects, classes etc)
- To understand how to define classes and objects using the Ada95 programming language
- To understand how to implement inheritance hierarchies in Ada95
- To understand what Abstract classes are and how to implement them using Ada95
- To understand what Polymorphism is and how to implement it using Ada95

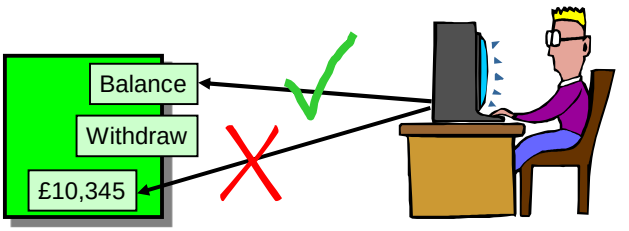
What is an Object?

- For the layperson
  - A tangible and visible thing
  - A person or thing seen as the focus for thought
- For the philosopher
  - That towards which cognition is directed
- For the linguist
  - A noun, pronoun or noun phase whose referent is the recipient of the action of a verb
- For the object technologist
  - An entity with a well defined boundary and identity that encapsulates state, represented by attributes, and behaviour represented by methods.



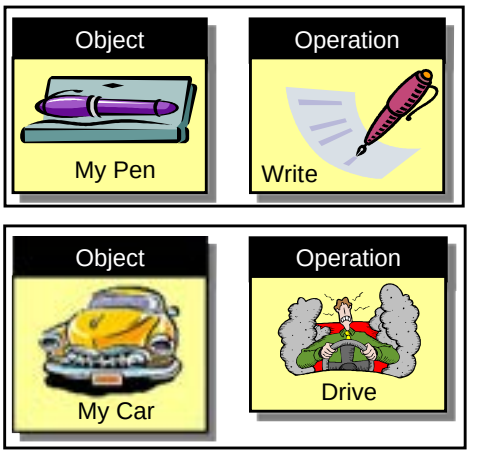
Objects Have State

- The state of an object is represented inside the object
  - Properties, data and relationships that an object knows about itself
  - An object's state is not directly accessible
- Objects are said to be *encapsulated*
  - The combination of behaviour and state in a single unit
  - The separation of a publicly visible interface from the privately maintained state



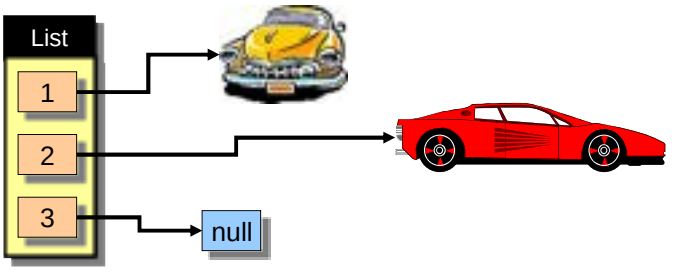
Objects Have Behaviour

- An object exists in order to provide behaviour
- Behaviour is defined by operations
  - The operations are expressed as a set of procedures and/or functions



Objects Have Identity

- Identity distinguishes one object from another
  - In a system, an object's identity is typically its physical address
- Pointers (access types) can reference objects indirectly



Object Summary

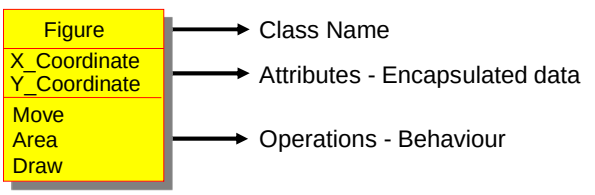
- An object has….
  - Identity that distinguishes it from others
  - A publicly visible interface, defined by its operations
  - Behaviour defined for each operation
  - Encapsulated state



What is a Class?

- A class defines a data type and operations on it
  - Name of the type
  - Defines the visible interface
  - Defines how the encapsulated data will be represented
- An object is an instance of a class
  - Users of a class create, manipulate and destroy objects of that type
  - State is modified and queried via operations defined by the class

UML representation of a class



Declaring Classes



Ada95 implements classes through tagged types:

Name of new data type

New reserved word

```
package Figure is
   type Object is tagged
   record
      X_Coordinate : Natural;
      Y_Coordinate : Natural;
   end record;
   function Area (This : access Figure) return Float;
   procedure Draw (This : access Figure);
   procedure Move (This : access Figure ;
                   New_X : in Natural;
                   New_Y : in Natural);
end Figure;
```

Attributes

Access Parameter

Public

Operations

Using Objects

- An object must be declared before use
  - Using package and class name as normal

Figure
X_Coordinate Y_Coordinate
Move Area Draw

Package with class definition

Object can be pointed at

```
with Figure;
procedure Main is
   MyFigure : aliased Figure.Object;
begin
   Figure.Draw (This => MyFigure'access);
   MyFigure. X_Coordinate := 1000;
end Main;
```

Object declaration

Call class operation

Modify object attributes
Should be illegal



Private Implementation

The reserved word ***private*** hides the implementation:

```
package Figure is

    type Object  is tagged private;
    function Area  (This: access Object) return Float;
    procedure Draw (This : access Object);
    …
private
    type Object is tagged
    record
        X_Coordinate : Natural;
        Y_Coordinate : Natural;
    end record;
end Figure;
```

Hides the representation

PRIVATE PROPERTY
NO TRESPASSING

Class users cannot access private parts

Benefits of Encapsulation

- Private data separates *use* from *implementation*
  - If the representation is changed….
  - The public interface often remains unchanged…..
  - Only the implementation of the primitive operators need changing

```
with Figure;
procedure Main is
    MyFigure  : Figure.Object;
begin
    Figure.Draw (This => MyFigure'access);
end Main;
```

Uses public interface

```
procedure Draw …….
private
type Object is tagged
record
```

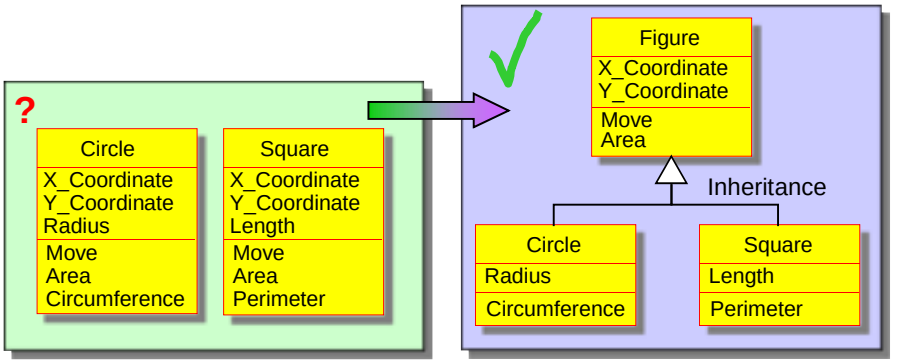
X_Coordinate : Natural;
Y_Coordinate : Natural;

X : Float ;
Y : Float ;

The public interface to the class is unchanged ….
In spite of a change of internal representation

Generalisation

- Generalisation is the process of factoring out common features in related classes into a single base class
  - Reduces duplication between similar classes
  - Increases productivity through reuse

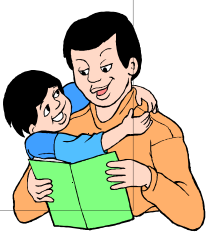


Inheritance

The difference between the OO and Object-Based approach is that the OO approach adds the concept of inheritance to mimic the way people think when they classify objects:

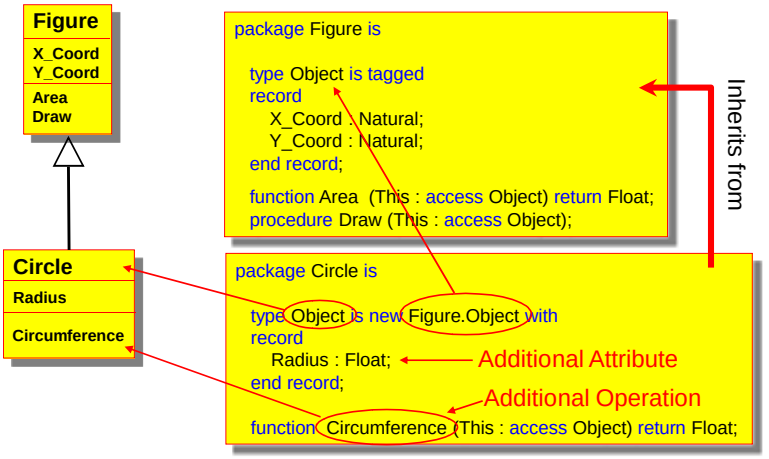
Inheritance

- Allows for the addition of new components
- Addition of additional subprograms
- Changes (overriding) of existing subprograms
- A class shares the structure and/or behavior of one or more classes



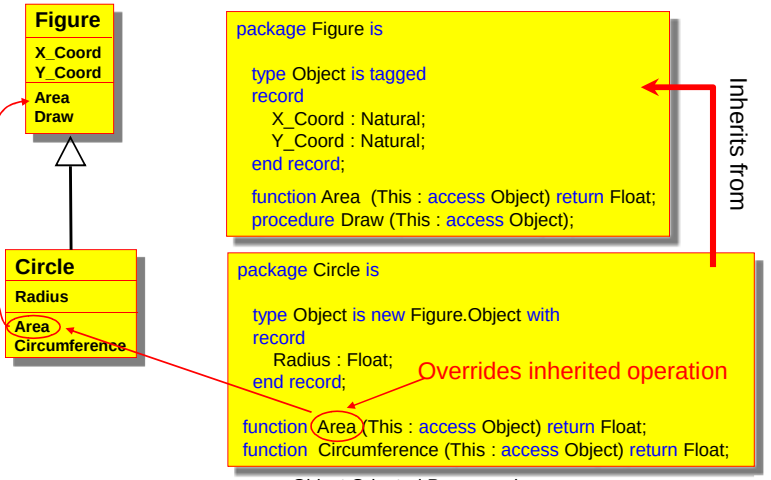
Inheritance Syntax

The syntax is straightforward, derive from the base type and extend:



Overriding Inherited Subprograms

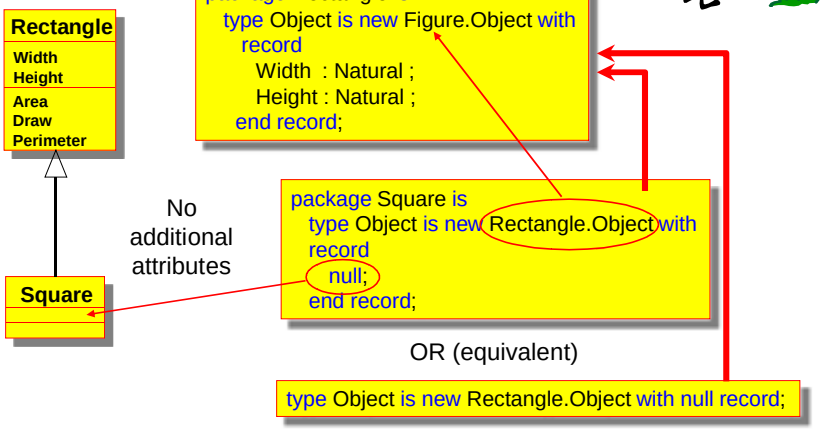
To override an inherited subprogram, we could write:



Null Extensions

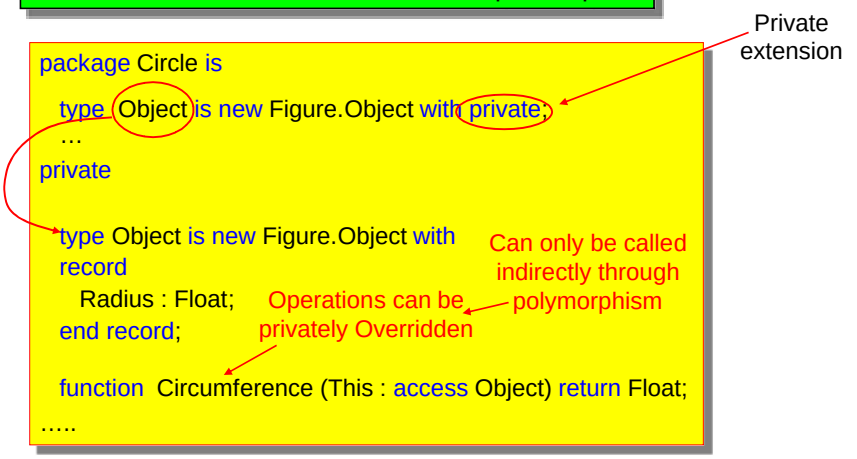
A class can be extended with no new components:





Private Extensions

An extension can also be hidden in the private part :



Protected Members

An additional access level is provided by the UML - Protected



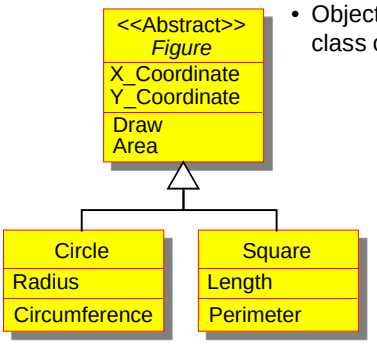
- UML notation for Protected
  - Any descendant of the class has visibility of the attribute or operation



- Child packages must be used if protected implementation is required
  - The class hierarchy matched with package hierarchy
  - The child package has visibility of the parents private parts!

Abstract Base Classes

An abstract class represents some generalised concept



- Objects of an abstract class cannot be declared

New reserved word for Ada95

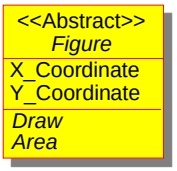
```
package Figure is
   type Object is abstract tagged
   record
      X_Coordinate : Natural;
      Y_Coordinate : Natural;
   end record;
   ……..
   Blob : Figure.Object ;
```

Will not compile

Abstract Operations

Subprograms can also be declared as abstract:

- No implementation for abstract operations
- When deriving from an abstract type, you must provide implementation for all of its abstract subprograms
- A call to an abstract operation would have to dispatch to an overridden version of the subprogram




```
package Figure is
   type  Object  is  abstract  tagged …..
   ……..
   function Area  (This : access Object) return Float is abstract;
   procedure Draw (This : access Object) is abstract;
```


Type Conversion

Type conversion is possible within the hierarchy:

```
…..
A_Figure : Figure.Object ;
A_Circle : Circle.Object ;
A_Rectangle : Rectangle.Object;
A_Square : Square.Object;
……
A_Figure := Figure.Object (A_Circle);

A_Rectangle := (A_Figure with
                    Height => 10.75 ,
                    Width  => 7.6);
A_Square := (A_Rectangle with null record);
```

This example assumes correct visibility

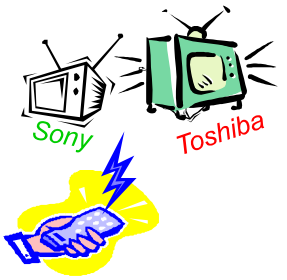
Always OK:
Components can only be added, never removed

Requires extension aggregate

Requires extension aggregate even if no extra components

Polymorphism

Allows different classes in the same inheritance hierarchy to be treated in exactly the same way as each other



- A single interface with multiple implementations
- The rule known as Dynamic Binding implies that the dynamic form of the object determines which version of the operation is applied
- Automatic run time decision making through class wide programming

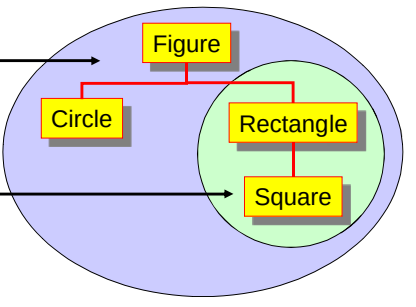
Class Wide Programming

Ada95 includes the concept of a class wide type:

For each tagged type T there is an associated type T'Class which comprises the union of all types in the tree of derived types rooted at T

Figure'Class is a class that includes the types Figure, Circle, Rectangle, and Square.

Rectangle'Class is a class that includes the types Rectangle and Square



Access to Class Wide Types

An access type can be declared which refers to a class wide type:

Figure_Handle can designate any object in the class (Figure, Circle, Rectangle or Square)

```
type Figure_Handle is access Figure.Object'Class;

Handle : Figure_Handle := new Circle.Object'
        (X_Coord    => 1,
         Y_Coord    => 5,
         Radius     => 10.75);
```

Dynamically creates and initialises a new circle

Dispatching

An operation can be defined to take a class wide parameter:

Class wide parameter

```
procedure Print_Area (This : in Figure_Class'Class) is
   The_Area : Float;
begin
   The_Area := Figure.Area (This => This);
   Ada.Text_IO.Put ("Area = ");
   Ada.Float_Text_IO.Put (The_Area);
end Print_Area;
```

Determine which Area subroutine to call based on the tag value

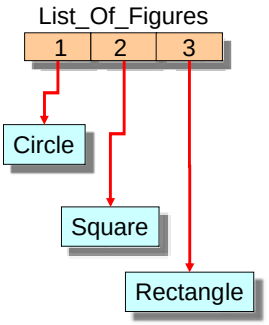
Primitive operation call (Dispatches)



A procedure with a class wide parameter is not a primitive operation and is never inherited

Dispatching

A Heterogeneous sequence of shapes now can be defined:



Class wide access type

```
type Figure_Handle is access Figure.Object'class;

List_Of_Figures : array (1..3) of Figure_Handle:=
   (1 => new Circle.Object' (…),
    2 => new Square.Object' (…),
    3 => new Rectangle.Object' (…) );
begin
   …….
```

Dispatching will occur when the pointer is de-referenced

Dispatching

Simply iterate over the list:

Primitive Call
The pointer is de-referenced and the correct version of draw is called

```
for Index in List'range
loop
      Figure.Draw (This => List (Index).all);
      Print_Area (This => List (Index).all);
end loop;
```

Classwide operation call

```
procedure Print_Area (This : in Figure_Class'Class ) is
   The_Area : Float;
begin
   The_Area := Figure.Area (This => This);
   Ada.Text_IO.Put ("Area = ");
   Ada.Float_Text_IO.Put (The_Area);
end Print_Area;
```

Determine which Area subroutine to call based on the tag value

Handling Tags

Ada95 provides a predefined package for handling tags:

- Expanded_Name returns the full expanded name of the type (identified by the tag) starting with the root library unit.

```
package Ada.Tags is
   type Tag is private;
   function Expanded_Name (T : in Tag) return String;

   function External_Tag (T : in Tag) return String;
   function Internal_Tag (External : in String) return Tag;

   Tag_Error : exception;
private
   -- language does not specify
end Ada.Tags;
```

Used for Stream I/O

Intentionally Left Blank

Controlled Types

Objective

- To understand the concept of Controlled types
- To understand how Package Ada.Finalization is used to define Controlled types
- To understand where controlled types could be used and what the disadvantages of using them are



The Concept

Controlled types provide the ability to provide procedures which are automatically called at the appropriate points in an object's lifetime



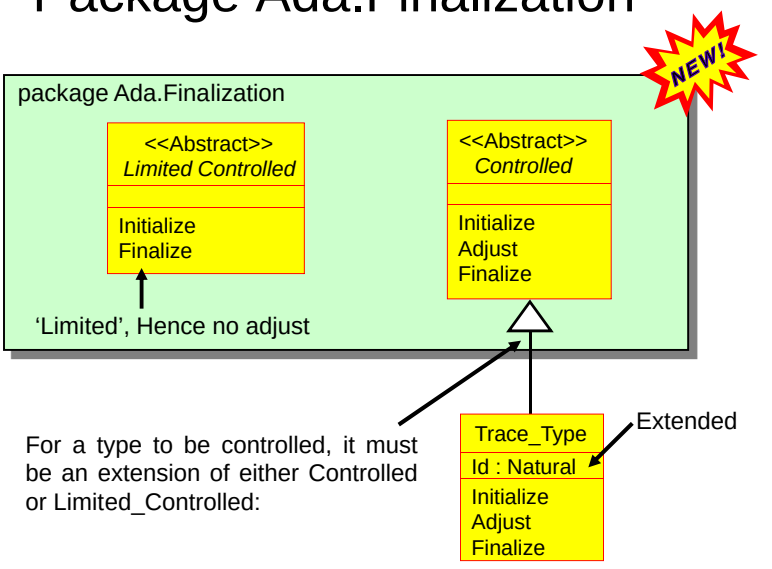
```
declare
   Object : Some_Type;
begin
   …
   Object := Another_Object;
   …
end;
```

Object is Created:
- Initialize (Object)

Object is Adjusted:
- Finalize (Object)
- Copy value
- Adjust (Object)

Object is Created:
- Finalize (Object)

Package Ada.Finalization



An Example Specification

```
with Ada.Finalization;

package Trace is

   type Trace_Type is new Ada.Finalization.Controlled with private;

   procedure Initialize  (Object : in out Trace_Type) ;
   procedure Adjust      (Object : in out Trace_Type) ;
   procedure Finalize    (Object : in out Trace_Type) ;

private
   type Trace_Type is new Ada.Finalization.Controlled with
   record
      Id : Natural := 0;
   end record;
end Trace ;
```

Derive a controlled type from Ada.Finalization

Overridden subprograms

Complete definition of Trace_Type which includes an Id for each object created

An Example Body

```
package body Trace is

   Next : Natural := 1;

   procedure Initialize (Object : in out Trace_Type) is
   begin
      Object.Id := Next;
      Next := Next + 1;
      Ada.Text_IO.Put_Line ("Initialize " & Integer'Image (Object.Id));
   end Initialize;

   procedure Adjust (Object : in out Trace_Type) is
   begin
      Ada.Text_IO.Put_Line ("Adjust " & Integer'Image (Object.Id));
   end Adjust;

   procedure Finalize .....
end Trace;
```

Keeps track of last number allocated

Assign the next object identifier and increment number allocated

Print the subprogram and object Identifier

Finalize (same as Adjust)

Main Program

```
with Trace;
procedure Test_Trace is
   Obj_1 , Obj_2 : Trace.Trace_Type;
begin
   Ada.Text_io.Put_Line ("Start of Block");
   declare
      Obj_3 , Obj_4 : Trace.Trace_Type;
   begin
      Obj_4 := Obj_3;
   end;
      Ada.Text_io.Put_Line ("End of Block");
      Obj_1 := Obj_2;
end Test_Trace;
```




```
Initialize 1
Initialize 2

Start of Block

Initialize 3
Initialize 4

Finalize 4
Adjust 3

Finalize 3
Finalize 3

End of Block

Finalize 1
Adjust 2

Finalize 2
Finalize 2
```


Uses for Controlled Types

Controlled types are particularly useful for:

- Controlled objects can be counted automatically (for example, a linked-list package could be made to count the peak number of nodes in existence for statistics gathering)
- Controlling object's allocation and deallocation (to implement automatic garbage collection)
- Implementing types whose "size" can vary without limit (these types are called "unbounded" types)



Ada83 Exceptions

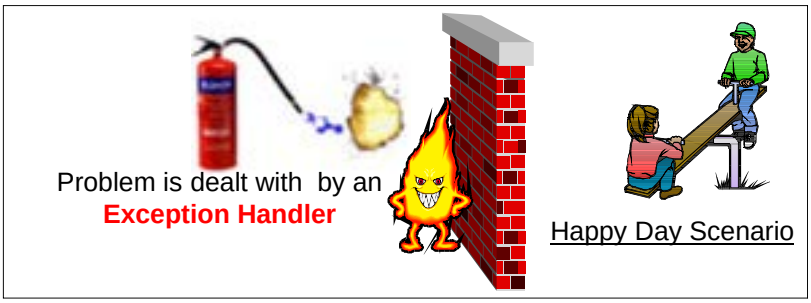
Objective

- To understand the concept of Exceptions
- To understand what the predefined exceptions that can be raised by an Ada83 program are
- To understand how to handle exceptions and how they are propagated
- To understand how to define and handle user defined exceptions



The Concept

No matter how experienced or how hard the programmer tries, it is a fact of life that rarely is a program perfect. As a program executes, it would be foolish to assume that nothing will go wrong:



Ada provides a mechanism by which programming errors (or the occurrence of exceptional events) can be contained and dealt with (or handled):

Predefined Exceptions

When an exception occurs, it is said to be "raised" and breaks the normal flow of the program.

There are 5 exceptions predefined by the Ada83 language:

Constraint_Error	-	Raised when something goes out of range
Numeric_Error	-	Raised when an arithmetic operation fails
Storage_Error	-	Raised if we run out of memory
Tasking_Error	-	Raised when the tasking rules are broken
Program_Error	-	Raised if the control structure of the language is violated

Handling Exceptions

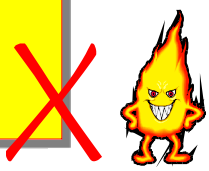
An Exception Handler is written to deal with exceptional situations:



Jump To Here

```
   Type Day is (Mon, Tue … Sun);
begin
   Tomorrow := Day'Succ (Today);
   Print (Tomorrow);
exception
  when Constraint_Error =>
    Tomorrow := Day'First;
end;
```

Constraint_Error if Today = Sunday



Bad Program

Exceptions should not be used by catch normal situations

Handling Exceptions

It is possible to handle a number of exceptions in the same exception block:



```
begin
   …  -- Statements                          "Constraint OR Numeric"
exception
   when   Constraint_Error |
          Numeric_Error      => Text_IO.Put ("Constraint/Numeric");
   when Storage_Error        => Text_IO.Put ("Storage");
   when others               => Text_IO.Put ("Something Else");
end ;
                  "Must be last and on its own"
```

Exceptions are Propagated

Exceptions are propagated until an exception handler is encountered:

```
function Divide (X,Y : in Integer) return integer is
begin
        return X / Y;
end Divide;
```

Divide by zero!!

Divide has no exception handler

```
Procedure Main is
   Result : Integer;
begin
   Result := Divide (X => 10 ,Y => 0);
exception
  when Constraint_Error    => Statements ;
  when others              => Statements ;
end Main;
```

Jump To Here



User Defined Exceptions

Ada provides a means of declaring exceptions that can be used to report specific erroneous conditions:



```
package Stack is

   Overflow   : exception ;
   Underflow  : exception ;

  type Stack_Type is …….

  procedure Push …….
  function Pop ……..

end Stack ;
```

Declared like a variable in the spec. and can therefore be handled by the calling routine.

Raising User Defined Exceptions

The exception Overflow can now be explicitly raised

```
package body Stack is

    procedure Push (Value ……...
    begin
          if Stack.Size = Max_Size
          then
                raise Overflow ;
          else
                Store value in Stack ...
```

Stack is full?

Raise problem

Jump Out Here





Exceptions should not be used by catch normal situations

Handling User Defined Exceptions

Exception raised in Stack can now be handled





```
with Stack ;
procedure Main_Program is
begin
    Stack.Push (Value => ……

exception
    when Stack.Overflow =>  Log_Error …..

    when others => ……

end Main_Program ;
```

If the stack is full

Jump To Here

Exception dealt with

Intentionally Left Blank

Ada95 Exceptions

Objective

- To understand the problems with Ada83 exceptions
- To understand how to elicit more information about exceptions using the Ada.Exceptions package
- To understand how to attach messages to exceptions in Ada95
- To understand the differences between Ada83 and Ada95 exceptions



Numeric Error is Obsolete

For compatibility reasons, the exception Constraint_Error is renamed as Numeric_Error in the package standard.




```
exception
   when  Numeric_Error | Constraint_Error => ….
```

Ada95 allows an exception to be mentioned more than once in the same handler, so allowing this without problem

Most Ada83 programs handle Numeric_Error and Constraint_Error together in the same handler, those that don't will no longer compile.

Handling Ada83 Exceptions



What went wrong ?



```
begin
   …. Statements

exception

   when others => Text_IO.Put ("Error ??");
   raise;

end;
```

No more information

We would really like to know more information about the exception.

Package Ada.Exceptions





New for Ada95, provides information about exceptions

```
with Ada.Exceptions;
……..
begin
    …. Statements

exception
    when  Event : others =>
      Text_IO.Put (Ada.Exceptions.Exception_Name (Event));
    raise;
end;
```

choice parameter



Returns a string

Exception_Message
Exception_Information

Attaching Messages

```
with Ada.Exceptions;
……..
declare
   Overflow : exception;
begin
   …
   Ada.Exceptions.Raise_Exception
                (E            =>  Overflow'Identity,
                 Message => "Trouble");
exception
   when  Event : others =>
     Text_IO.Put (Ada.Exceptions.Exception_Message (Event));
```


Attribute identifies exception



returns message

Saving Exceptions

Ada.Exceptions provides the facility to enable exception occurrences to be saved for later analysis




```
procedure Save_Occurrence (Target  : out Exception_Occurrence;
                           Source : in  Exception_Occurrence);
```

Limited private type

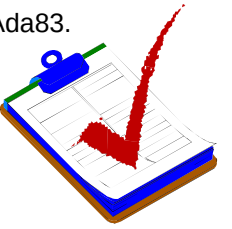
```
function Save_Occurrence (Source : in Exception_Occurrence)
    return Exception_Occurrence_Access;
```

Which copies the source parameter to a target parameter, implementation is free to truncate the message to 200 characters.

Summary of Changes

The following list summarises the changes introduced by Ada95:

- Numeric_Error and Constraint_Error were distinct exceptions in Ada83.
- The same exception was not allowed twice in a handler in Ada83.
- Exception occurrences were not available in Ada83.



Ada83 Tasking

Objective

- To understand the concept of tasking and parallel processing and the problems it introduces
- To understand how to suspend tasks using the 'delay' statement
- To understand various task scheduling models and how tasks communicate
- To understand how to define and implement tasks using Ada83, including task types



Parallelism

Procedures execute in sequential order:

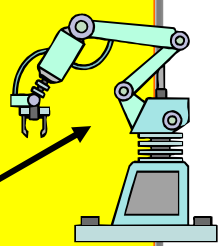
```
procedure Sequential_Arm is

   procedure Move_Shoulder (Angle : in Integer) is
   begin
      ...
   end Move_Shoulder;

   procedure Move_Elbow ….
   procedure Move_Wrist ….
begin
   Move_Shoulder (Angle => 10);
   Move_Elbow (Angle => 45);
   Move_Wrist (Angle => 60);

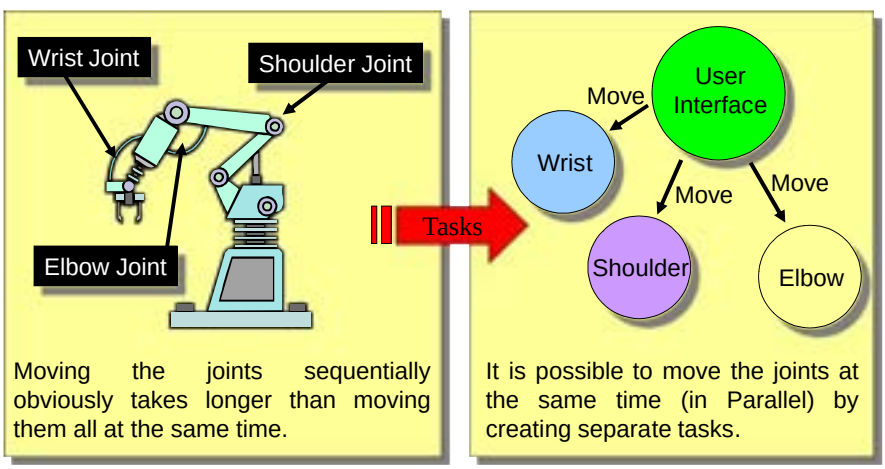
end Sequential_Arm;
```

Sequential



Parallelism

Sometimes we want to execute threads in parallel :



Moving the joints sequentially obviously takes longer than moving them all at the same time.

It is possible to move the joints at the same time (in Parallel) by creating separate tasks.

Ada Tasks

Tasks can be considered as “separate programs” used to carry out the tasks in parallel:

Every task, like a package, consists of a specification describing the interface:

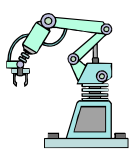
```
task Shoulder is
   entry Move_To (Angle : in Integer);
end Shoulder;
```

And a body describing the dynamic behaviour:

```
task body Shoulder is
begin
   accept Move_To (Angle : in Integer)….

end Shoulder;
```


Ada Tasks



Now, the three movements can be performed in parallel:

3 threads of control

```
task Shoulder is
   entry Move (Angle : in Integer);
end Shoulder;
```

```
task Elbow is
   entry Move (Angle : in Integer);
end Elbow;
```

```
task Wrist is
   entry Move (Angle : in Integer);
end Wrist;
```

```
begin
   Shoulder.Move (Angle => 10);
   Elbow.Move (Angle => 45);
   Wrist.Move (Angle => 60);
end Tasking_Arm;
```

The tasks can be executed via a rendezvous

Scheduling

More complicated systems such as a control system employing multiple sensors and actuators, execute at different rates:

Flight Control System

Every 20 milliseconds
- **Read sensor data**
- **Validate sensor data**
- **Compute pitch / yaw / roll**

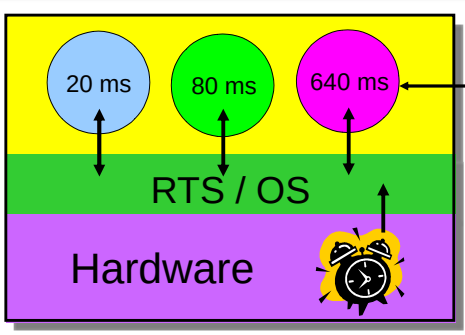
Every 80 milliseconds
- **Update tracking data**
- **Keyboard & mode selections**
- **Perform Stress Calculations**



Every 640 milliseconds
- **Execute Built-In-Test**
- **Check for failed RTs**

Scheduling

An Ada program, supported by either a Run-Time System (RTS) or Operating System (OS):



Executable Program Structure

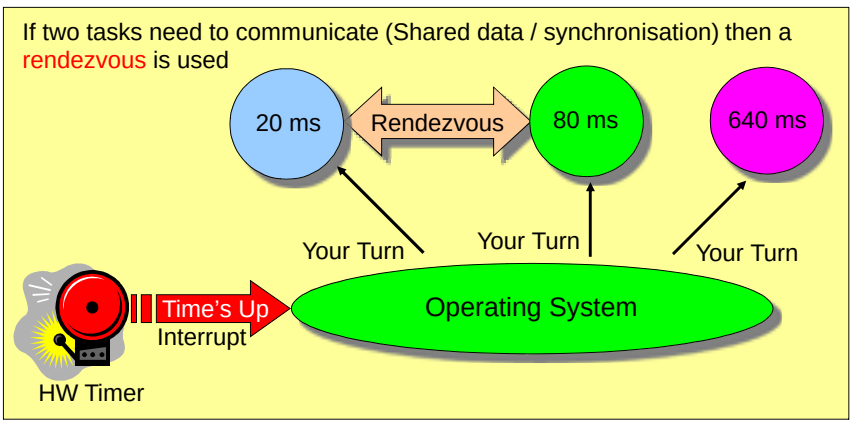
Each task can be thought of as an individual program performing the steps necessary to fulfil its requirement.

If the tasks need to communicate, they can do so via rendezvous.

The RTS / OS schedules the tasks at the required time.

Scheduling

A hardware clock normally tells the RTS /OS when it is time for the next task to be scheduled for execution:



The *delay* Statement

It is often necessary to suspend the current task:

- It may be necessary to perform timing within a task
- It may be necessary to suspend a task for a period of time
- It may be necessary to allow other equal priority tasks to run, if not operating in a time-slicing environment



```
delay 3.0;
```

Predefined type Duration

Suspends the current task for at least three seconds!!

Priorities

Each task can be assigned a priority:

- The priority is a number indicating the relative urgency of a particular task
- Priorities help the operating system allocate processors to tasks when there are more tasks ready to execute than available processors

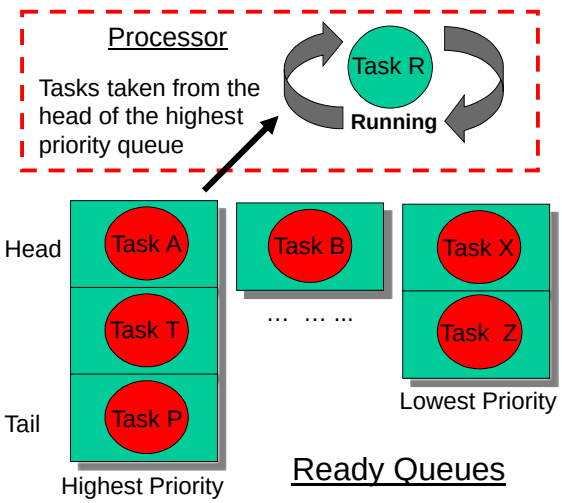



```
task Wrist is
   pragma priority (7) ;
   entry Move (Angle : in Integer);
end Wrist;
```

Implementation dependant

The Run-Time System determines which tasks to execute based on their priority

The Task Dispatching Model



Ready queues are ordered list of tasks that are ready to run.

Each processor has one ready queue for each priority value. At any instant, each ready queue contains exactly the set of tasks of that priority that are ready for execution

Simple Periodic Task

A simply delay statement inside a loop:

To print 'Step' no more often than every second:

```
with Text_IO;
task Periodic;
task body Periodic is
begin
   loop
      Text_IO.Put_Line ("Step");
      delay 1.0;
   end loop;
end Periodic;
```

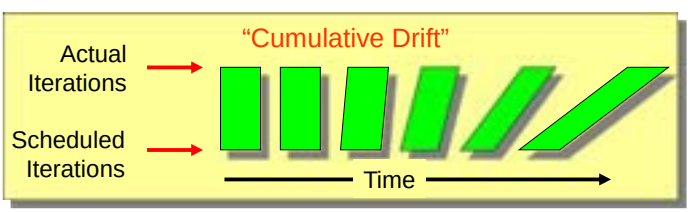
Simply delay (at least) one second in each iteration of the loop.

The expression after the reserved word delay, is of a predefined fixed point type

Cumulative Drift

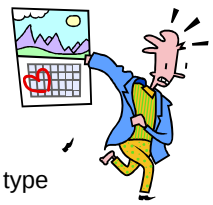
Although this solution is simple, it is unsatisfactory for two reasons:

- We have not taken account of the execution time of the printing or the overhead of the loop.
- When a tasks delay expires, the operating system might not allocate the processor to that task immediately" this can cause:



Little by little, the task falls further behind, it can not catch up, the effect is cumulative

Package Calendar



Can be used to achieve more sophisticated timing:

```
with Calendar;
.....
task Periodic;
task body Periodic is
   Wake_Up_Time : Calendar.Time;
   Period :  constant Duration := 1.0;
begin
   loop
      Wake_Up_Time := Calendar.Clock + Period;
      Text_IO.Put_Line ("Step");
      delay Wake_Up_Time - Calendar.Clock;
   end loop;
end Periodic;
```

Private type

Clock returns the current time

Delay expiry time takes the time to print and the overhead of the loop into account

Rendezvous



Entries are used to synchronise with a task:

Known as a rendezvous point

Task Specification

```
task Shoulder is
   entry Reset;

   entry Move (Angle : in Float);
end Shoulder;
```

Parameterless Entry

Entry with one 'in' parameter



Entries may have 'in', 'out' or 'in out' parameter; there is no concept of an entry returning a value like a function

Rendezvous



Accept appears in the task body:

```
task body Shoulder is
begin
   accept Reset;
   -- Reset joint goes here...
   loop
      accept Move (Angle : in Float)
      do
            Move Shoulder Commands…
      end Move;
   end loop;
end Shoulder;
```

Accept must match entries in spec

Executed in the rendezvous

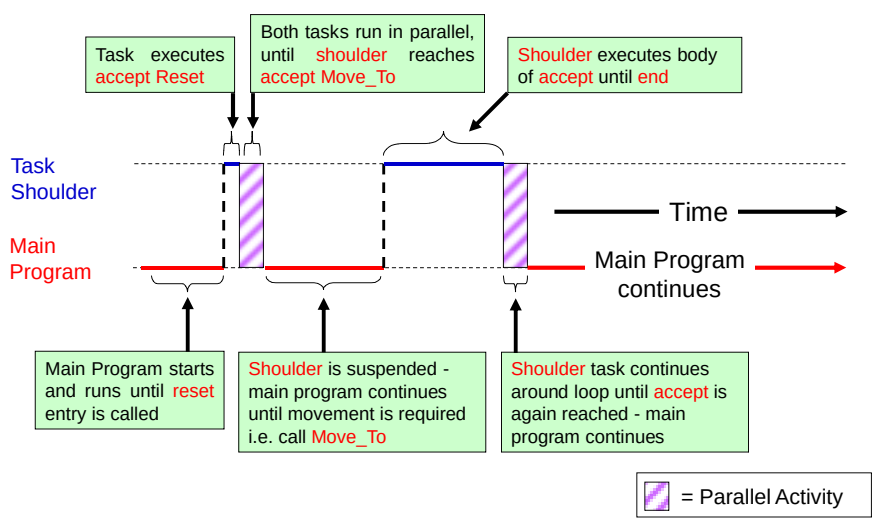
Accept statement without associated block

Accept statement with 'do' block

Rendezvous is not complete until here

Parameter goes out of scope at 'end'

Rendezvous Timeline



More Parallel Processing

The execution must be moved out of the rendezvous:

```
task body Shoulder is
   Copy_Of_Angle : Float;
begin
   accept Reset;
   -- Reset joint goes here...
   loop
      accept Move (Angle : in Float)
      do
            Copy_Of_Angle := Angle;
      end Move;

      -- Move Shoulder Commands…
   end loop;
end Shoulder;
```

Task level variable; visible within task body

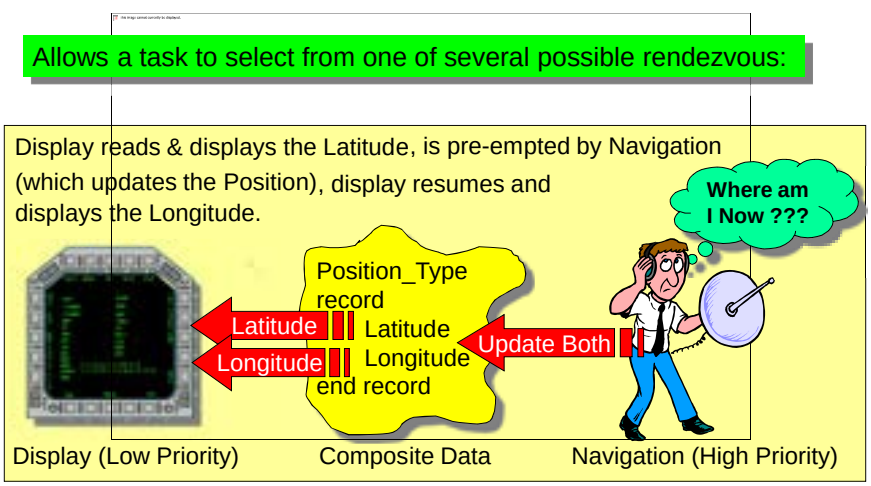
Accept body simply copies the parameters

Rendezvous complete - caller is permitted to continue

Movement continues in parallel

The *Select* Statement

Allows a task to select from one of several possible rendezvous:



The data displayed is **inconsistent !!**

Select Statements

The solution to the problem is to use a task with entry calls:

```
task Position is
   entry Read
      (Value : out Position_Type);
   entry Write
      (Value : in Position_Type);
end Position;
```

```
task body Position is
   Present_Position : Position_Type;
begin                          Composite type
   loop
      select
         accept Read (Value : out Position_Type) do
            Value := Present_Position;
         end Read;                    Return current value
      or
         accept Write (Value : in Position_Type) do
            Present_Position := Value;
         end Write;                   Update current value
      end select;
   end loop;
end Position;
```

Select statement allows task to wait for either Read or Write

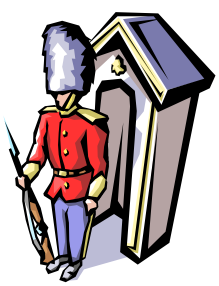
Implementing Guards

Guards control which rendezvous are allowed:

```
task body Position is
   Present_Position : Position_Type;
   Written_To : Boolean := False;
begin
   loop
      select
         when Written_To =>
         accept Read (Value : out Position_Type)
         do
            Value := Present_Position;
         end Read;
         ....
```

The guard must be TRUE before the service is offered

Write would have to set Written_To = True



Select Statement with a Timeout

Select statement can contain a delay statement alternative:

```
select
   accept Read (Value : out Position_Type) do
      Value := Present_Position;
   end Read;
or
   accept Write (Value : in Position_Type) do
      Present_Position := Value;
   end Write;
or
   delay 0.020;
   Present_Position := Default_Value;
end select;
```

When the select statement is executed, the delay timeout is started

If the delay expires before a rendezvous is executed, the timeout processing is executed

If neither Read nor Write are called within 20 milliseconds, the timeout processing is executed.

Select Statement with an Alternative

Another form of select statement is one that contains an else part:

```
select
   accept Read (Value : out Position_Type) do
      Value := Present_Position;
   end Read;
or
   accept Write (Value : in Position_Type) do
      Present_Position := Value;
   end Write;
else
   -- alternative processing
   Perform_Alternative_Processing;
end select;
```

If no tasks are waiting to rendezvous with either Read or Write when the select is entered, then the alternative processing specified in the else part is executed immediately.

Timed Out Entry Calls

Select statements may also be used when calling a task entry:

- Allowing a sequence of statements to be executed if the call is not accepted within a specified duration:

Write is an entry call on the Position task

If the task Position does not allow an entry to write within 40 milliseconds, the statements following the delay will be executed.

Start delay

```
select
   Position.Write (New_Position);
or
   delay 0.040;
   -- Perform Timeout Processing
end select;
```



Tentative Rendezvous

Also known as a conditional entry call:



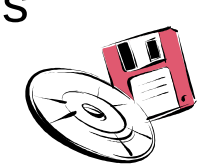
- Allowing a sequence of statements to be executed if the call is not accepted immediately

Write is an entry call on the Position task

```
select
   Position.Write (New_Position);
else
   -- Perform Alternative Processing
end select;
```

If the task Position is not in a state allowing it to accept the entry call write, the caller will immediately perform the alternative processing

Declaring Task Types



It is also possible to declare tasks as types:

- For Ada83, the storage size of a task can be set ONLY if the task is declared as a task type.

```
task type Queue_Type is
   entry Write (Value : in Integer);
   entry Read (Value : out Integer);
end Queue;

Queue_1 : Queue_Type;
Queue_2 : Queue_Type;
begin
Queue_1.Write (Value => 995);
```

A task type is declared by including type in the specification.

An instance of a task is declared as a variable declaration



The body remains unchanged from a task object

Ada95 Tasking

Objective

- To understand the main differences introduced into the Ada95 tasking model
- To understand how to solve cumulative drift and how to suspend tasks using the ‘delay until’ statement
- To understand how to protect shared data using protected objects
- To understand the rules that apply to protected objects



Exceptions



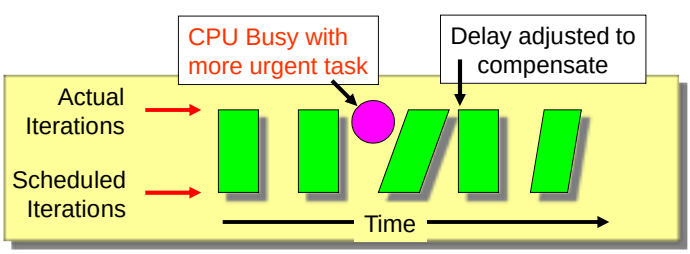
Accept statements can now have their own exception handlers:

```
task body Shoulder is
begin
    accept Reset;
    loop
        accept Move_To (Angle : in Float) do
            -- Move Shoulder Commands…
        exception
            when others => Reset_Joint;
        end Move_To;
    end loop;
end Shoulder;
```

Exceptions can now be dealt with locally.

Solving Cumulative Drift

By calculating the Absolute time a task should be scheduled.



These small deviations from the scheduled start time is known as "Jitter"

The *delay until* Statement

The delay until construct takes the period as an Absolute time:

- Reduces drift as the expiry time of the delay will be exact

```
    Next_Iteration : Calendar.Time := Calendar.Clock;
    Period : constant Duration := 1.0;
begin
    loop
        delay until Next_Iteration;
        Do_Activity;
        Next_Iteration := Next_Iteration + Period;
end loop;
```



For Ada95

Takes an absolute time

The *abort* Statement



New form of select statement which comprises two parts:

- The triggering statement
- Abortable part

Both execute in parallel, which ever finishes first causes the other to be abandoned.

```
select
    delay 10.0;
    Use_The_Default_Data;
then abort
    Get_Data_From_The_Interface;
end select;
```

Triggering statement

- Either a delay statement or an entry call

If getting the data takes longer than 10 seconds, it is aborted

Task Types with Discriminants

Ada95 allows task types to have discriminants:



- The discrimimant can be either an access type or a discrete type
- The body of the task can use the discriminants in the normal way

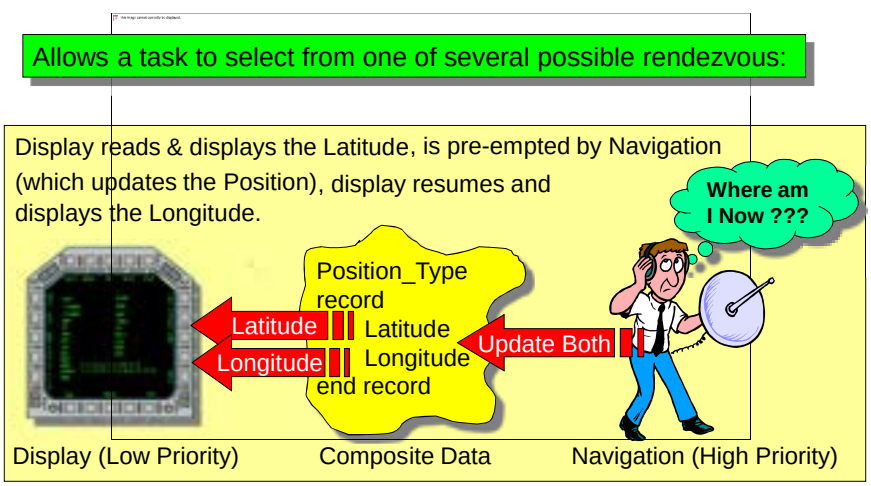
```
type Activity_Type is access procedure;
task type  Do_Activity (The_Activity  : Activity_Type ;
                        Number_Of_Times    : Positive);
```



- Discriminants are passed in when the object is declared
- Characterises the task with the activity to perform and the number of times to perform it.

The *Select* Statement

Allows a task to select from one of several possible rendezvous:



The data displayed is **inconsistent !!**

Protected Objects

Provide mutual exclusion to shared data:



- A protected object has two distinct parts
  - Specification - Access protocol & private data
  - Body - Implementation

```
protected Present_Position is
    procedure Write (Value: in Position_Type);
    function  Read return Position_Type;
private
    The_Position : Position_Type;
end Present_Position;
```

Read/Write so mutual exclusion is guaranteed

Read only and so multiple reads can occur concurrently

Data declared in the private part of the spec

The Protected Object Body

The corresponding body would look like:

- Unlike packages, no data can be declared in the body

```
protected body Present_Position is

    procedure Write (Value : Position_Type) is
    begin
        The_Position := Value;
    end Write;

    function  Read return Position_Type is
    begin
        return The_Position;
    end Read;

end Present_Position;
```

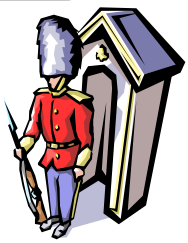
A procedure can access the protected data in an arbitrary manner

A function is only allowed read access

Guards on Protected Objects

Guards can also be placed on access to protected objects

- Protected objects can guard against data being read before it is written to through entry points



```
protected Stack is
    procedure Push (Value : in Integer);
    entry Pop (Value : out Integer);
private
    The_Stack : Some_Stack_Type;
    Count : Natural := 0;
end Stack;
```

Entries are used instead of functions

Protected Data

Guards on Protected Objects

Entries always have an associated boolean expression:

```
protected body Stack is

    procedure Push (Value : in Integer) is
    begin
        -- Write the value to the stack
        Count := Count + 1;
    end Push;

    entry Pop (Value : out Integer)
        when Count > 0 is
    begin
        -- Read the Value from the Stack
        Count := Count - 1;
    end Pop;

end Stack;
```

A call is blocked until the guard is evaluated to TRUE

The entry call will only proceed when the condition is evaluated to be true i.e. the Count is greater then 0

Protected Types

Like tasks, Protected objects can be defined as types:



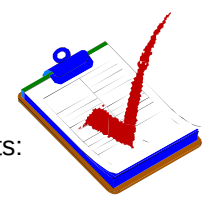
- Insert the reserved word type in the specification
- The body remains unchanged

```
protected type Stack_Type is
    procedure Push (Value : in Integer);
    entry Pop (Value : out Integer);
private
    ....
```

```
declare
    My_Stack : Stack_Type;
begin
    My_Stack.Push (Value => 748);
    My_Stack.Pop (Value => My_Value);
```

Protected types are definite, meaning they can be array components

Protected Rules



There are a number of rules applied to protected objects:

- A protected operation may **NOT** call another protected operation on the same object, this could cause deadlock.
- The body of a protected function **CAN** call other protected functions on the same object but not protected procedures.
- A protected procedure **CAN** call an internal protected subprogram (functions or procedures) or an external protected subprogram on a different target object.
- When a protected object is deallocated, any remaining entry calls are removed and a program_error is raised at the point the call was made.

Access to Protected Operations

An access type can also refer to a protected operation:




```
protected Stack_Type is
   procedure Push (Value : in Integer);
   function Pop return Integer;
private
   …
```

Access type that can point at protected operation, signature must match

```
type P_Access is access protected procedure (V : in Integer);
type F_Access is access protected function return Integer;

The_Procedure  :  Procedure_Access := Stack.Push'access;
The_Function   :  Function_Access := Stack.Read'access;
```

The operations are called indirectly in the normal way

The *Requeue* Statement

New for Ada95 is the requeue construct which allows:



- The servicing of calls in an entry queue to be deferred until later
- The redirection of a call to another entry or task
  - (Ada83 would have caused deadlock if the entry was in the same task)
- An accept statement that calls an entry in another task to complete without waiting for the call to return

A requeue can be on:

```
requeue Reset;                          -- The same object
requeue Buffer.Store;                   -- A named object
requeue Activity_List (This).Execute;   -- A named object etc.
```

The *Requeue* Statement

Consider the following (taken from Ada95 by John Barnes):

```
protected Event is
    entry Wait;
    procedure Signal;
private
    Occurred : Boolean := False;
end Event;
```

```
protected body Event is

    entry Wait when Occurred is
    begin
        Occurred := False;
    end Wait;

    procedure Signal is
    begin
        Occurred := True;
    end Signal;

end Event;
```

This kind of event is known as persistent since an event is remembered until a task issues a wait.

This type of event is similar to a binary semaphore.

The *Requeue* Statement

Another form of signal is the broadcast signal:



- Allows ALL current tasks to proceed when the event occurs, then the event must be forgotten.
- Must prevent tasks getting through, that call wait after the event has occurred, but before the signal can be reset

```
protected Event is
    entry Wait;
    entry Signal;
private
    entry Reset;
    Occurred : Boolean := False;
end Event;
```

Private entry
- can only be called from entries within the protected object.

The *Requeue* Statement

The corresponding body would then be:

```
protected body Event is

   entry Wait when Occurred is
   begin
      null;
   end Wait;

   entry Signal when True is
   begin
      if Wait'Count > 0
      then
         Occurred := True;
         requeue Reset;
      end if;
   end Signal;

   entry Reset when Wait'Count = 0 is
   begin
      Occurred := False;
   end Reset;
end Event;
```

- Wait has a null body so calling tasks can suspend
- If Wait has tasks, it releases them and regains control so that it can reset the flag
- At the end of the body of a protected entry, all the barriers are re-evaluated for entries that have tasks queued.

Interfacing

Objective

- To understand how to define representation for scalar and enumeration types
- To understand how to define representation for arrays and records
- To understand how to do unchecked programming using unchecked conversion
- To understand how to handle interrupts and how to map data to specific memory locations



Representation

When compiling a program, the compiler is free to determine how the various data types (and objects) are represented

```
type Byte_Type is range 0..2**8-1;
Object_1 : Byte_Type := 0 ;
Object_2 : Byte_Type := 0 ;
```

Range is always 0 to 255

Can occupy 8, 16, 32 or even 64 bits

The compiler will normally align objects for easy addressing:

- Byte boundary
- Word boundary
- Long Word boundary



Representing Scalar Types

A representation clause directs the compiler to store the specified types (and objects) as the programmer requires

‘Size attributes represents the object size in Bits

```
type Byte_Type is range 0..2**8-1 ;
for Byte_Type'Size use 8;

Object_1 : Byte_Type := 0 ;
Object_2 : Byte_Type := 0 ;
```

Range is always 0 to 255

Objects of Byte_Type Will always occupy exactly 8 bits

Representing Enumeration Types

The representation can also be defined for enumeration types

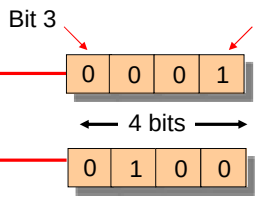
```
type Mode_Type is (Off , Standby, Test, On);
for Mode_Type'Size use 4;
for Mode_Type use ( Off      => 2#0001# ,
                    Standby  => 2#0010# ,
                    Test     => 2#0100# ,
                    On       => 2#1000# );
```

Literal values

Object size = 4 bits

Underlying representation

```
The_Mode : Mode_Type := Off ;
.....
begin
      The_Mode := Test ;
```

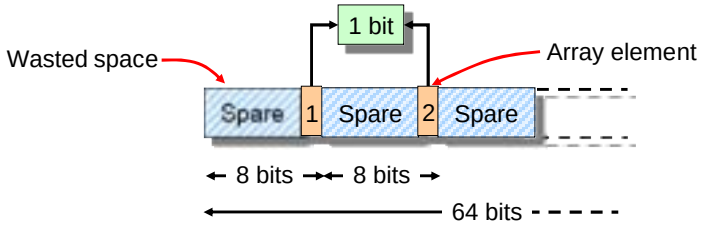


Representing Arrays

The representation can also be defined for arrays, consider:

```
type Array_Type is array (1..8) of Boolean;
My_Array : Array_Type;
```

Could possibly result in using 8 * 8 bits (64 bits) when 8 would do:



Representing Arrays

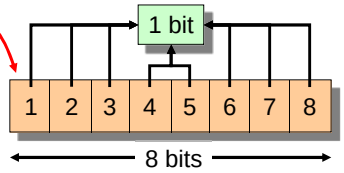
pragma, Pack, ensures the compiler does not leave any space:

```
type Array_Type is array (1..8) of Boolean;
pragma Pack (Array_Type);
for Array_Type'Size use 8;
```

Pragma Pack indicates that the array should be tightly packed

Size attribute indicates that object of this type should only occupy 8 bits

NO wasted space



Less efficient at run time due to bit manipulation

Representing Records

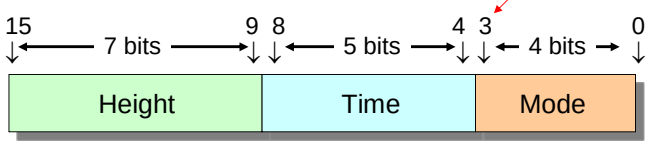
Without representation, the compiler is free to store the record components in any order:

```
type Record_Type is
record
   Mode   : Mode_Type;
   Time   : Time_Type;
   Height : Height_Type;
end record;
```

Representation

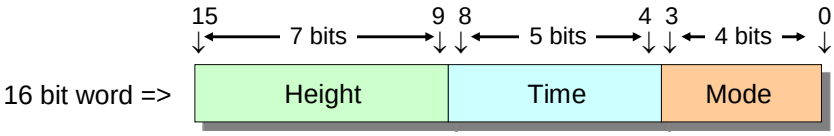
Byte number from start of data

```
for Record_Type use
record
   Mode   at 0 range 0 .. 3;
   Time   at 0 range 4 .. 8;
   Height at 0 range 9 .. 15;
end record;
```



Unchecked Programming

Often we will be given a document to describe H/W interface or the message format between two computers, processors etc:



It would be tedious to write:

This is very long winded and is extremely unmaintainable →

```
if Raw_Data = 2#0001#
then
   Mode := Off

elsif Raw_Data = 2#0010#
then
   Mode := Standby;
…
```

Due to strong typing, we cannot copy a 16 bit word to a record

Unchecked Programming

Unchecked_Conversion performs a bit pattern copy :

The source and target must be the same size

```
generic
      type Source (<>) is limited private;
      type Target (<>) is limited private;
function Ada.Unchecked_Conversion (S:Source) return Target;
```

The function can be instantiated:

Copies raw data to typed form, no type checking

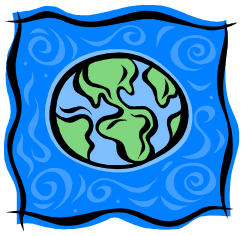
```
with Ada.Unchecked_Conversion;
…..
function To_Message is new Ada.Unchecked_Conversion
      (Source => Sixteen_Bit_Word_Type,
       Target => Record_Type);
…..
Converted_Message := To_Message (S => Raw_Data);
```

Mapping To Hardware

An object can be mapped to a specific hardware address:

```
Radar_Message : Message_Type;
for Radar_Message'Address use 16#FFFE_0000#;
```

- The mapping of objects to an address is implementation dependant.
- Care should also be taken to range check the values of the converted or mapped data.
- A Constraint_Error will be raised if the value is out of range !!!



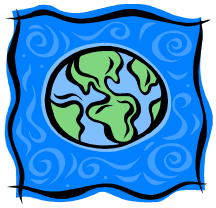
Mapping To Interrupts

It is possible to link an entry point of a task directly to an interrupt:

```
task Interrupt_Handler is

    entry Handle;
    for Handle use at 16#400#;

end Interrupt_Handler;
```



- The entry 'Handle' is entered when interrupt vector 400 Hex is generated by the hardware.
- Implementations often place restrictions on what can appear within the interrupt handler. Refer to the implementation documentation

Variant Record Types

Objective



- To ensure an understanding of Ada record structures
- To understand how to define records with variable parts (variant records)
- To understand how to define variant records with a defaulted discriminant
- To understand the rules that apply to variant records

What are Records?

Sets of logically grouped components, typically of different types:

- Data is often grouped together, eg
  - Dates can be represented DD/MM/YY
  - A person can be described by name, date of birth, etc
- A new composite type can be defined using a *record*
  - record corresponds to *struct* in C and C++'s



```
type Date is
record
      Day, Month , Year : Natural ;
end record;
```

```
type Person is
record
      Name : String (1..5);
      DOB : Date ;
end record ;
```


Variant Record Types

Have both a fixed part and a variable part:



Parameter

```
type Employee_Type (Contractor : Boolean) is
record
      Phone_No : Phone_No_Type;
      case Contractor is
            when True  =>  Hourly_Pay    : Float;

            when False =>  Annual_Pay    : Float;
                           Days_Holiday  : Natural;
      end case;
end record;
```

All employees have a telephone number

Only Contractors get paid hourly

Only staff get paid annually and get holidays

Variant Record Types

When an object is declared, it must be constrained:

```
This_Wally : Employee_Type (Contractor => True);
```



- A subtype could be introduced which means objects can then be declared without a constraint:

```
subtype Permy_Type is Employee_Type (Contractor => False);
subtype Subby_Type is Employee_Type (Contractor => True);
.....
This_Permy  : Permy_Type ;
This_Subby  : Subby_Type ;
```

These can never change employee types!!!

Variant Record Types

Aggregation should be used when assigning values to objects

```
This_Subby := (Contractor   => True,
               Phone_No     => 0385_740200,
               Hourly_Pay   => 25.0);
```



- The value applied to Contractor MUST be static so that the compiler can check the consistency of the aggregate
- Once an object is declared to be of a certain constraint, it cannot be changed, (once a Subby, always a Subby)

Variant Record Defaults

Allows objects to be declared without a constraint

Default makes the type definite

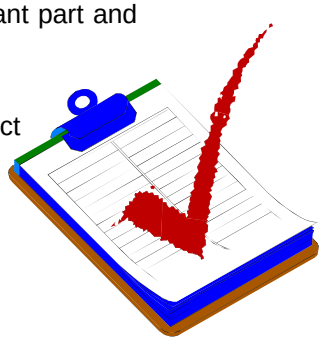
```
type Employee_Type (Contractor : Boolean := False) is
record
      Phone_No : Phone_No_Type;
      case Contractor is
            when True  => Hourly_Pay    : Float;
            when False => Annual_Pay    : Float;
                          Days_Holiday  : Natural;
      end case;
end record;
```

Now This_Permy can mutate into a Subby.

```
This_Permy : Employee_Type;
```


Variant Record Rules

- A variant record can only contain one variant part and it must be last
- All components of the record must be distinct
  - Even in different alternatives
- If an object is declared with a constraint, it cannot be changed



- Attempting to access a wrong component will result in a Run-Time error being raised (Constraint_Error)

Unconstrained Array Types

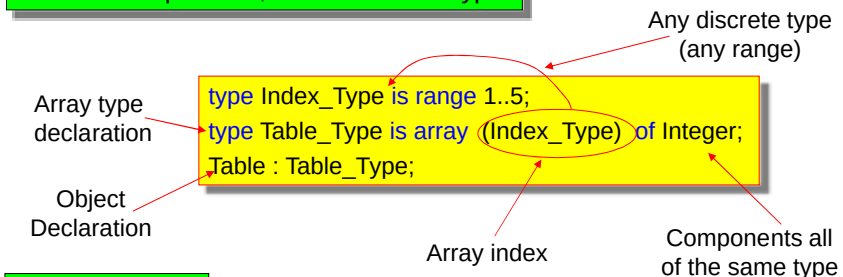
Objective



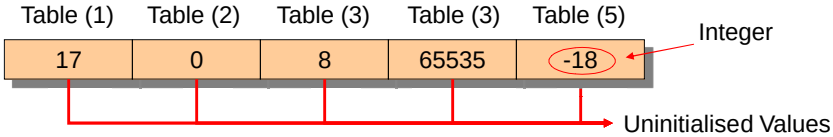
- To ensure an understanding of Ada array and multiple dimension array structures
- To understand how to define arrays of varying length (unconstrained arrays)
- To understand how to concatenate two arrays together
- To understand the rules that apply to unconstrained arrays

What is an Array?

A set of components, all of the same type:



Would result in:



Multi-Dimensional Arrays

Have several dimensions, a range is provided for each dimension:

Object : array (1..4 , 1..5) of Float ;

The components can then be accessed as before but with the addition of another dimension:

Object (2 , 4) := 0.0 ;

	1	2	3	4	5
1	1.7	4.2	7.6	4.1	9.6
2	9.7	1.1	4.0	0.0	2.3
3	3.6	6.4	2.9	10.2	41.1
4	8.4	0.8	21.8	13.9	12.1

The range of the index can be variable (i.e. not static) :

```
Index : Integer := 6;
Object : array (Integer range 1 .. Index) of Float;
```

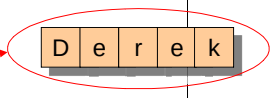


Arrays of Arrays

An array component can be any definite type:

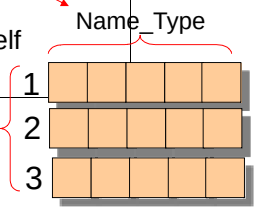
- A definite type being one for which an object can be declared

```
subtype Name_Type is String (1..5);
Name : Name_Type := "Derek";
```



- A fixed length array is a definite type:
  - Therefore can be an array component itself

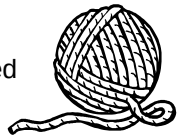
```
type People is array (1..3) of Name_Type;
```



Unconstrained Arrays

The constraints for the indexes are not explicitly given:

- Type *String* is an unconstrained array of characters
- Unconstrained arrays are indefinite types
- An object of an unconstrained array cannot be declared
  - Unless it is bounded



```
type String is array (Positive range <>) of Character;
```

Predefined type

Unconstrained notation

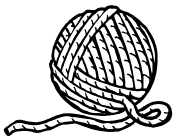
```
Hacker : constant String := "Derek";
Author : String (1..5);
Error : String ;
```

Size = 5, bounds derived from the initial value

Explicit bounds

No bounds given

Concatenation



Can be used to append arrays together:

Bounded by initial value

```
Word_1 : constant String := “Happy” ;
Word_2 : constant String := “Birthday” ;
Greeting : constant String  :=  Word_1  &  “ “  &  Word_2;
```

String with value of 1 space

Concatenation

Strings can be printed directly using Text_IO:



```
Text_IO.Put (“Hello Folks”) ;
Text_IO.Put_Line (Greeting) ;
```

Text String

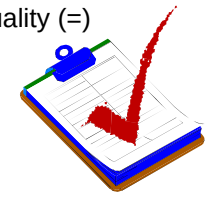
Object; String

Unconstrained Arrays Notes

- When assigning unconstrained types:
  - The source and destination must be of the same type
  - The number of components MUST be identical
  - The bounds may differ

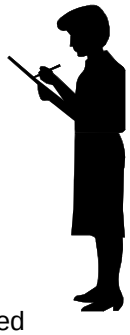
```
type Array_Type is array (Integer range <>) of Integer;
Initial_Values : constant Array_Type (1..5) := (0, 1, 2, 4, 8) ;
Object : Array_Type (0..4)  := Initial_Values;
```

- Two unconstrained arrays can be compared for equality (=)
  - *True* will only be returned if:
    - Every component has the same value
    - The dimensions are the same



Discriminated Record Types

Objective

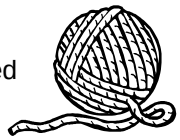


- To ensure an understanding of Ada unconstrained array structures
- To understand how to define records containing arrays of varying length (discriminated records)
- To understand how to apply defaults to discriminated records

Unconstrained Array Recap

The constraints for the indexes are not explicitly given:

- Type *String* is an unconstrained array of characters
- Unconstrained arrays are indefinite types
- An object of an unconstrained array cannot be declared
  - Unless it is bounded



```
type String is array (Positive range <>) of Character;
```

Predefined type

Unconstrained notation

```
Hacker : constant String := "Derek";
Author : String (1..5);
Error : String ;
```

Size = 5, bounds derived from the initial value

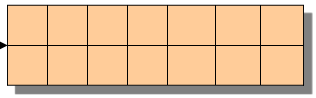
Explicit bounds

No bounds given

Discriminated Record Types

Is a method by which a parameter can be applied to a type:

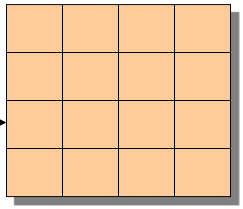
```
type Matrix_Type is array
   (Natural range <> ,
    Natural range <>) of Integer;
```



Objects can have unequal bounds natural * natural

To create objects that are always square, we could write:

```
type Square_Type (Size : Natural) is
record
   The_Square : Matrix_Type (1..Size , 1..Size);
end record;
```



Will always be square

Discriminated Record Types

Variables and constants can now be declared of type Square_Type:

```
Square_1 : Square_Type (Size => 4) ;
Square_2 : Square_Type (Size => An_Object) ;
```

Static expression

Dynamic expression

Objects can also be initialised when declared:

```
Default_Square : Square_Type :=
  (Size            => 4 ,
   The_Square => ( 1..4 => ( 1..4 => 0 )));
```

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

Once an object is declared, its constraints cannot be changed

Discriminated Record Defaults

Allows objects to be created and change size dynamically:

```
subtype Index_Type is Natural range 0..20;
type Square_Type (Size : Index_Type := 0) is
record
   The_Square : Matrix_Type (1..Size , 1..Size);
end record;

Mutable_Square : Square_Type;
```

Squares can now be any size from:
1 X 1 to 20 X 20

Can be null (1..0)

Initially Size = 0

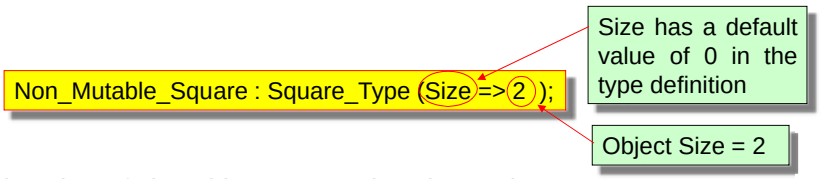
The size of the object can only be changed by a complete record assignment, so:

```
Mutable_Square.Size := 4;
```

is illegal

Discriminated Record Defaults

An initial size can also be provided for Objects declared of discriminated records that have defaults applied:



The size of the object cannot be changed, even though a default is applied, so

```
Non_Mutable_Square :=
      (Size           => 3
      The_Square =>(1..3 => (1..3 => 0 )));
```

is illegal

Ada95 Predefined Library

Objective



- To understand the concept of the predefined library environment for Ada95
- To understand the facilities provided by the predefined packages ‘Standard’ & System
- To understand how to interface with other languages using package ‘Interfaces
- To understand how to perform character and string manipulation using the predefined library
- To understand the facilities provided by the predefined packages ‘Numerics’ & ‘Text_IO’

Predefined Library

A main objective of Ada95 was to supply a set of supplemental packages of general utilities in order to promote portability and reusability.

The predefined library is structured as three packages:

- System
- Interfaces
- Ada

These packages can be thought of as being child packages of package standard, each containing numerous child packages.

Every implementation of Ada contains the predefined library.

Package Standard

Package Standard can be thought of as a special package containing the implementation specific values of entities such as:

- Boolean
- Integer (Natural & Positive)
- Float
- Character (Wide Character)
- String (Wide String)
- Duration
- Predefined Exceptions (Constraint , Program , Storage , Tasking)
- Predefined Operators

Package Standard should not be thought of as a package that actually exists.

Package System

Package System deals with the control of storage and other memory related issues. It is structured as a number of child packages:

- System.RPC **(Distributed Systems not covered here)**
- System.Storage_Elements
- System.Address_To_Access_Conversions
- System.Storage_Pools
- System.Machine_Code **(Used to incorporate machine code inserts)**

Package System contains a number of system-dependant named numbers, e.g. Max_Int , Min_Int, Max_Binary_Modulus etc.

Types and constants are also provided corresponding to memory, e.g. Address, Storage_Unit, Word_Size, Memory_Size etc.

System.Storage_Elements

The child package System.Storage_Elements defines facilities for manipulating addresses and offsets. Its specification includes:

- Storage_Element - Type represents a storage element
- Storage_Array - Type represents a contiguous block of memory
- Integer_Address - Type for representing actual addresses
- “+” , “-” operators - Allows addresses and offsets to be calculated

There are also functions which enable addresses to be converted to Integer_Address and vice-versa:

- To_Address
- To_Integer

System.Address_To_Access_Conversion

Package Address_To_Access_conversion is a generic package that allows the programmer to ‘Peek’ and ‘Poke’ at memory locations.



The use of these packages is extremely dangerous and care should be taken to avoid erroneous situations.

System.Storage_Pools

When access types are used to allocate objects, it is important that the objects are deallocated when they are finished with to prevent storage leak.

Ada implementations do not typically have garbage collectors, so it is important to control the allocation and deallocation of objects.

The child package System.Storage_Pools defines facilities for creating storage pools in a fully controlled manner.

Users must define a type derived from Root_Storage_Pool and provide concrete subprograms for Allocate, Deallocate and Storage_Size which will automatically Allocate and Deallocate whenever storage is required or released.

Interface Pragmas

Package System provides a set of packages and some special pragmas to interface with other computer languages. The three most useful pragmas are:

- Pragma import - imports a subprogram from another foreign language
- Pragma export - exports an Ada subprogram to a foreign language.
- Pragma Convention - specifies that a specified type should use the storage conventions of a given foreign language.

Interface Pragmas

The following example shows how an Ada program might attach a callback to a C program:

```
type Callback_Type is access procedure (Value : Value_Type);
pragma Convention (C, Callback_Type);

procedure Attach (Callback : Callback_Type);
pragma Import (C , Attach);

procedure Response (Value : Value_Type) is separate;
pragma Convention (C, Response);
…
Attach (Response'access);
```

External to the Ada program

Ada procedure, parameter convention is 'C'

Attach Ada procedure to a 'C' callback

Package Interfaces

Package Interfaces contains a number of child packages which contain facilities for communication between Ada programs and other programming languages such as C, COBOL and Fortran:

- Interfaces.C
- Interfaces.C.Strings
- Interfaces.C.Pointers
- Interfaces.COBOL
- Interfaces.Fortran

Interfacing to machine code can be achieved one of two ways:

- package Machine_Code - Embedded machine code instructions (painful)
- Pragma Import - Call assembler routine directly

Package Ada

Most of the predefined library is structured as child packages of Ada which is empty except for the pragma Pure (which specifies that the package does not have any library-level state)

There are numerous child packages for handling things such as:

- Characters and Strings
- Command line
- Input / Output
- Unchecked programming
- Tasking facilities
- Exception handling

Package Ada.Characters

There are three child packages of Ada for character handling:

Ada.Characters	-	Simply declares itself pure
Ada.Characters.Handling	-	Character classification and conver
Ada.Characters.Latin_1	-	Contains constant declarations

Ada.Characters.Handling contains the following classification and conversion functions:

•Is_Control	•Is_Basic	•To_Lower (characters & strings)
•Is_Graphic	•Is_Digit	•To_Upper (characters & strings)
•Is_Letter	•Is_Decimal_Digit	•To_Basic
•Is_Lower	•Is_Special	•Is_String (True if not wide chars)
•Is_Upper	•Is_Hexadecimal_Digit	•To_String
•Is_ISO_646	•Is_Alphanumeric	•To_Wide_String

Package Ada.Characters

Package Ada.Characters.Latin_1 is a child package of Ada.Characters and contains declarations of constants giving names to most of the characters so they can be referred to as:

- Ada.Characters.Latin_1.Pound_Sign
- Ada.Characters.Latin_1. Registered_Trade_Mark_Sign
- ….
- ….
- Ada.Characters.Latin_1.Left_Curly_Bracket
- Ada.Characters.Latin_1. Degree_Sign

There are also a number of alternatives for the same character, e.g. Minus_Sign for Hyphen.

Package Ada.Strings

Ada95 provides an extensive range of string handling facilities:

- Search for patterns in strings
- Search for characters in program-specified sets
- Translation (via a character-to-character mapping)
- Transformation (replacing, inserting, overwriting)
- Deleting of substrings

Through a set of child packages:

- Ada.Strings
- Ada.Strings.Fixed
- Ada.Strings.Bounded
- Ada.Strings.Unbounded
- Ada.Strings.Maps
- Ada.Strings.Maps.Constants

Types of String

Ada95 provides 3 different types of strings:

- Fixed - Individual objects of fixed strings all have a fixed length which can not change
- Bounded - Have a maximum length but of which only a part is in use at any time, i.e. The string can dynamically change its length
- Unbounded - Have no maximum length (actually they are bounded by integer'last)

Package Ada.Strings

Package Ada.Strings.Bounded is a generic package which must be instantiated with the maximum length of the bounded string:

```
package Ada.Strings.Bounded is
   generic
      Max : Positive;
   package Generic_Bounded_Length is
      Max_Length : constant Positive := Max;
      type Length_Range is Natural range 0..Max_Length;
      type Bounded_String is private;
      …
      function Length (Source : in Bounded_String) return Length_Range;
      ...
   end Generic_Bounded_Length;
end Ada.Strings.Bounded;
```

Maximum length of the bounded string

Package Ada.Strings

Bounded strings must be created using an instantiation of Ada.Strings.Bounded.Generic_Bounded_Length:

```
with Ada.Text_IO;
with Ada.Strings.Bounded;

procedure Print_My_Aircraft is
      package Bounded is new
         Ada.Strings.Bounded.Generic_Bounded_Length (Max => 80);
      Aircraft : Bounded. Bounded_String;
begin
      Aircraft := Bounded.To_Bounded_String (“Harrier”);
      Ada.Text_IO.Put (Bounded.To_String (Aircraft));
end Print_My_Aircraft;
```

Instantiation for strings up to 80 characters

Declaration of Bounded string

Setup and print bounded string

Package Ada.Strings

Package Ada.Strings.Unbounded.Unbounded_String is non-generic and is used to declare Unbounded strings:

```
with Ada.Text_IO;
with Ada.Strings.Unbounded;

procedure Print_My_Aircraft is
        Aircraft : Ada.Strings.Unbounded.Unbounded_String;
begin
        Aircraft := Ada.Strings.Unbounded.To_Unbounded_String ("Harrier");
        Ada.Text_IO.Put (Ada.Strings.Unbounded.To_String(Aircraft));
end Print_My_Aircraft;
```

Declaration of Unbounded string

Set-up and print Unbounded string

There is no direct conversion between Bounded and Unbounded strings

Package Ada.Strings

The following list some of the more useful string operations:

To_(Un)bounded_String converts string into type (Un)bounded_String

To_String converts (Un)bounded_String into type String

Length returns the number of characters currently stored in the string

Append appends a string or (un)bounded string to (Un)bounded string

Element extracts a character from a (Un)bounded_String at a given position

Replace_Element lets you modify a character at a given position

Slice returns a "slice" of an (Un)bounded string (in type string)

Insert inserts a string into an (Un)bounded_String at a given position

Delete deletes the characters within a String between two indexes

Package Ada.Strings

The following examples show a small number of the string facilities available. The Ada RM Section A.4 provides a complete definition of Ada.Strings:

```
The_String : Unbounded_String := To_Unbounded_String ("  Harrier");

The_String := Trim (The_String , Left);                    -- “Harrier”
The_String := Replace_Slice (The_String , 1 , 1, "B");     -- “Barrier”
The_String := Delete (The_String , 4, 7);                  -- ”Bar”
The_String := Insert (The_String , 4, " Maid");            -- “Bar Maid”
The_String := Overwrite (The_String , 4, " Tender");       -- “Bar Tender”
The_String := Translate  (The_String , Upper_Case_Map);    -- “BAR TENDER”
```

To aid readability, it assumes that the appropriate Packages have been ‘use’d:

Ada.Strings.Maps

The last example on the previous slide (Translate) enabled a string to be converted using a translation known as character mapping. In addition to the mapping provided, users can write their own translation mappings function (the signature must match):

```
type Character_Mapping_Function is access
    function (From : in Character) return Character;

function Convert (C : in Character) return Character is
begin
      if Is_Lower (C) then
            return To_Upper (C);
      else
            return C;
      end if;
end Convert;

Translate (“Bae” , Convert’access);  -- “BAE”
```

Ada.Strings.Maps

Character_Mapping tables provide another method of converting strings. This is achieved by calling:

```
function To_Mapping (From , To : Character_Sequence)
  return Character_Mapping;
```

Character_Sequence is a rename of String (for readability). From and To must be the same length. Characters in **From** map on to characters in the same position of **To**.

```
Language_Map : Character_Mapping := To_Mapping ("{}[]" , "()()");
Translate ( "{if I was you} [I would give up]" , Language_Map);
```

would result in (if I was you) (I would give up)

Convert all brackets to ()

Ada.Strings.Maps

Some of the string handling functions (Index, Count, Find_Token and Trim) can take parameters of type Character_Set. We can define our own characters sets by using the function **To_Set** which can take:

```
Set_1 := To_Set (';');                    -- A single semicolon
Set_2 := To_Set ("AEIOU");                -- String, any vowel
Set_3 := To_Set (('3' , '7'));            -- Range, any number
Set_4 := To_Set ((('A' , 'Z') , ('a' , 'z')));  -- array of 2 ranges
```

The function Index could now be called with the set passed in, so:

```
The_String := To_Unbounded_String (" 0357zaAB; ");
Index (The_String , Set_1)      -- Index = 9   (; is at position 9)
Count (The_String , Set_2)      -- Count = 1   (only 1 capital A)
Index (The_String , Set_3)      -- Index = 2   (digit 3 is at position 2)
Count (The_String , Set_4)      -- Count = 4   (zaAB)
```

Package Numerics

The library package Numerics is the parent of several child units that provide facilities for mathematical computation:

- Numerics. Discrete_Random
- Numerics. Float_Random
- Numerics. Generic_Elementary_Functions

The package Numerics itself contains 3 elements:

- Constant definition of pi
- Constant definition of e
- Argument_Error - An exception that is raised by child packages if one or more of the actual subprogram parameters are outside the domain of the corresponding mathematical function

Generic_Elementary_Functions

The package Ada.Numerics.Generic_Elementary_Functions has a single generic type parameter suited to any floating point type:

```
generic
   type Float_Type is digits < > ;
package Ada.Numerics.Generic_Elementary_Functions…..
```

It provides the following functions:

- Sqrt, Log, Exp, “**”
- Sin, Cos, Tan, Cot
- Arcsin, Arccos, Arctan, Arccot
- Sinh, Cosh, Tanh, Coth
- Arcsinh, Arccosh, Arctanh, Arccoth

Random Numbers

Package Ada.Numerics.Float_Random provides a limited private type Generator which is a pseudo-random number generator returning floating-point values. The package contains the following facilities:

- Random (Read next random number)
- Save (Save the current random number)
- Image (Return the string representation of the number)
- Value (Converts a string back to the state)
- Reset (Time, integer or last saved value)

The package Ada.Numerics.Discrete_Random is almost the same as that for Float_Random but the generic parameter is Result_subtype;

Random Numbers

To generate random numbers, a Generator is declared, it is initialised to an unspecified state that does not vary from one program execution to another.

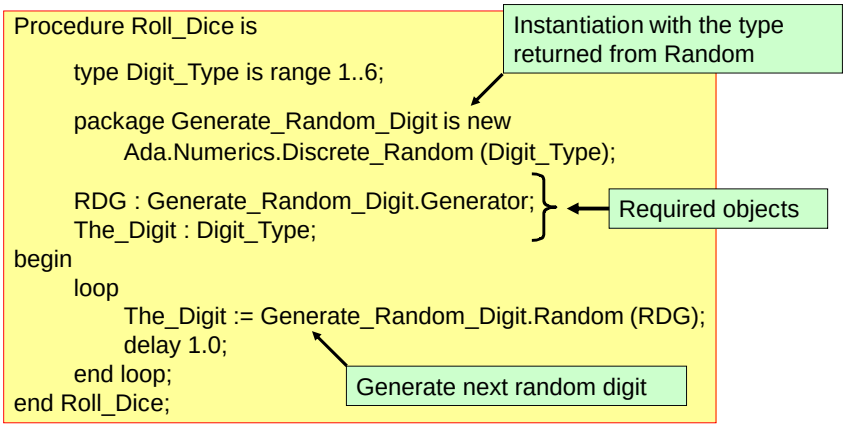
The next random number in the sequence can be obtained by calling the function Random:

```
G : Ada.Numerics.Float_Random.Generator ;
X  : =  Random (Gen => G) ; --  also moves G on to the next state
```

The generator can be reset to a time-dependent state, Saved and Restored

Random Numbers

An example random number generator for rolling dice:



Intentionally Left Blank